

Modeling and Analyzing the Influence of Non-Item Pages on Sequential Next-Item Prediction

ELISABETH FISCHER, ALBIN ZEHE, ANDREAS HOTHOTH, and DANIEL SCHLÖR, Data Science Chair, Center for Artificial Intelligence and Data Science (CAIDAS), Julius-Maximilians-Universität (JMU), Germany

Analyzing sequences of interactions between users and items, sequential recommendation models can learn user intent and make predictions about the next item. Next to item interactions, most systems also have interactions with what we call non-item pages: these pages are not related to specific items but still can provide insights into the user's interests, as, for example, navigation pages. We therefore propose a general way to include these non-item pages in sequential recommendation models to enhance next-item prediction.

First, we demonstrate the influence of non-item pages on following interactions using the hypotheses testing framework HypTrails and propose methods for representing non-item pages in sequential recommendation models. Subsequently, we adapt popular sequential recommender models to integrate non-item pages and investigate their performance with different item representation strategies as well as their ability to handle noisy data. To show the general capabilities of the models to integrate non-item pages, we create a synthetic dataset for a controlled setting and then evaluate the improvements from including non-item pages on two real-world datasets.

Our results show that non-item pages are a valuable source of information, and incorporating them in sequential recommendation models increases the performance of next-item prediction across all analyzed model architectures.

CCS Concepts: • **Computing methodologies** → *Neural networks*; • **Information systems** → **Recommender systems**.

Additional Key Words and Phrases: sequential recommendation, next-item prediction, non-item pages

ACM Reference Format:

Elisabeth Fischer, Albin Zehe, Andreas Hothoth, and Daniel Schlör. 2025. Modeling and Analyzing the Influence of Non-Item Pages on Sequential Next-Item Prediction. 1, 1 (February 2025), 40 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Sequential recommender models are useful building blocks for providing personalized recommendations to users. By analyzing the sequence of historical interactions of the user with items, sequential recommendation models can learn and infer user preferences and make predictions about the next item of interest. Recently, especially deep learning approaches based on recurrent neural networks (RNNs) [11], convolutional neural networks (CNNs) [29], and Transformer models [16, 26] have received attention, typically modeling each interaction with an item as one step in the sequence. Significant amounts of research have been conducted that incorporates additional item information, for example, textual and visual item features [12, 31], item attribute embeddings [8, 19], and categorical and numerical

Authors' Contact Information: Elisabeth Fischer, elisabeth.fischer@informatik.uni-wuerzburg.de; Albin Zehe, zehe@informatik.uni-wuerzburg.de; Andreas Hothoth, hothoth@informatik.uni-wuerzburg.de; Daniel Schlör, schloer@informatik.uni-wuerzburg.de, Data Science Chair, Center for Artificial Intelligence and Data Science (CAIDAS), Julius-Maximilians-Universität (JMU), Würzburg, Germany.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

Manuscript submitted to ACM

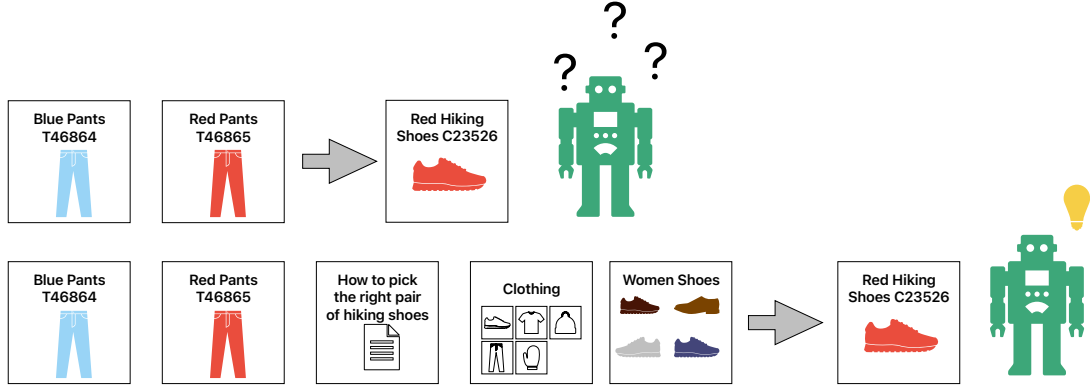


Fig. 1. Exemplary click sequence in an online store. The inclusion of non-item pages provides an explanation for the shift in user interest, enabling a more accurate prediction of their preferences.

item features [4]. Apart from click sequences on item pages, most systems utilizing recommenders also incorporate additional forms of information. This includes data about users and pages that do not directly represent an item but still involve a user interaction. Although there are many papers exploring strategies to incorporate user-specific information [1, 8, 29, 34], the aspect of leveraging information from non-items in sequences to further improve recommendation has been less explored. However, in many applications, interactions with pages that are not items themselves or cannot even be associated with a particular item (hereafter referred to as non-item pages, cf. Section 3.2) still provide useful information in the context of sequential recommendation. Examples of these pages include search results or blog posts that contain descriptions of entire item categories, which can be very helpful in detecting the user’s intent when browsing through a website. Figure 1 shows a case where including these non-item pages can be useful: When only looking at the item history, we see that the user switches from pants to hiking shoes without any explanation. Including non-item pages, we see that the user is interested in hiking equipment, which explains their next step of looking at hiking shoes and enables the recommender to specifically suggest other hiking shoes as opposed to more pants.

We argue that ignoring this information, which is available in most real-world settings (cf. Section 3.2), limits the potential of recommender systems to follow the user’s intent. To verify our intuition that non-item pages provide valuable information, we analyze our datasets with the HypTrails framework [25]. We formalize and test our hypothesis that non-item pages influence the following interaction, finding that this is indeed the case. To leverage this information, we propose incorporating non-item pages and present various modeling approaches for non-item pages for sequential next-item prediction for a variety of commonly used recommender models. As a proof of concept for our approach in a controlled setting, we create a synthetic dataset with artificial non-item pages, which are highly related to the following item. We additionally study the influence of noisy non-item pages with varying degree of relatedness in this setting. Finally, we evaluate the effectiveness of our modeling on two real-world datasets from the e-commerce domain.

We compare different types of non-item page representations (based on their content or a unique id) and evaluate eight popular sequential recommender models on click sequences enriched by these representations. Specifically, we consider three types of non-item pages: (1) a list of items, (2) a single non-item, e.g. a blog post, and (3) a list of non-items.

To cover a wide variety of popular or recent architectures, we adapt several models in the RecBole [41] framework: GRU4Rec, a popular RNN baseline model, CASER and NextItNet as models based on CNNs and several models based on the attention mechanism: NARM combining RNNs and attention, SASRec as the first Transformer-based model, and BERT4Rec adapting bidirectional Transformers. We also include CORE, which couples the encoding and decoding representation spaces and LightSANs, which introduces a low-rank decomposed self-attention. We extend our evaluation by shedding light on the specific representations of items and non-items across different models and representation types, including an analysis of item embedding similarities across various configurations.

In summary, our proposed modeling approaches and evaluation address the gap in current research regarding the modeling of non-item pages in sequential recommendation systems and show the utility of including such information.

To sum up our contributions, we:

- 1) validate our hypothesis that non-item pages influence subsequent interactions and
- 2) propose various modeling approaches for non-item pages.
- 3) We adapt a wide set of models to include categorical and vector representations of pages in a generic way and
- 4) analyze the capabilities of different architectures to include these on one synthetic and two real-world datasets.

Our results show that non-items can reveal valuable information about the user’s intent, but also the importance of finding good representations for non-item pages. All adapted models are capable of taking advantage of the additional non-item pages, depending on the representations, showing the general applicability of our approach.

This paper is a direct extension of our work in [7], which included only SASRec and BERT4Rec. We expand our work by generalizing the approach and applying it to a wider set of sequential recommender architectures in a popular recommender framework. We expand our experiments to examine the behavior of models on noisy data and provide deeper insights on the effects of different models and embedding variants on the resulting item representations. Our results confirm previous findings and underline the general utility of including non-item pages.

The remainder of our paper is structured as follows: Section 2 provides an overview of related work. We introduce the concept and proposed modeling approaches for non-item pages in Section 3, as well as our proposed adaptations to sequential recommender models to include them. In Section 4 we present the datasets, while Section 5 covers our experiments. We start by testing our hypotheses about non-item pages, before analyzing their inclusion in sequential recommenders in different setups. We discuss some of the general findings and compare them with the results of our previous paper in Section 6. Our conclusion and closing remarks can be found in Section 7.

2 Related Work

In this section, we first present an overview on deep sequential recommendation and then focus on architectures handling information apart from the sequence of item interactions themselves. With the increasing popularity of neural networks, neural architectures have been widely employed for modeling and learning sequences of user interactions [11, 16, 18, 26, 29, 36]. While these works focused on the dependencies on a per-item level, some studies aimed at incorporating additional information along with the items, which can be sorted into two specific groups, either adding additional information for items or adding additional information to model the user.

The inclusion of additional item information has been the goal of several modifications of the models mentioned above. Hidasi et al. [12], for example, model textual and visual item features in separate RNNs and combine their output afterward, while Tuan et al. [31] include character-wise item descriptions using CNNs with 3D convolutions. NovaBert [19] uses bidirectional transformers and merges item attributes directly in the attention mechanism as query

and value, as do the authors of [35]. Fischer et al. [8] allow the integration of categorical item attributes in BERT4Rec by adding attribute embeddings to the item embeddings, while AttribERT [15] uses attribute dictionaries to represent products. Transformers4Rec [4] uses models like XLNet [37] or Electra [2] from the Natural Language Processing domain and integrates categorical and numerical item features. Liu et al. [20] show how the inclusion of context (being time and location) can be utilized in a RNN.

Regarding user information, several approaches have been proposed, for example, CASER [29], SSE-PT [34], the work of Chen et al. [1] and TARN [40], which combines user, item, and temporal context. They treat the user separately from the sequence, while the approach by Fischer et al. [8] integrates the user representation as the first item in a sequence. This is more similar to our usecase, but in contrast to user representations, interactions with non-item pages can appear several times and have specific positions in the sequence.

Some works that participated in the SIGIR Coveo Challenge 2021 [27] are notable exceptions, as they include additional information that is not related to items or users. The Coveo dataset contains page views that are not related to items, as well as search results. Three participants in the challenge, [9, 14, 22], embed page views as an item in the sequence for transformer models, while [24] uses the count of page views and the first page view as features for an ensemble model of LSTMs and Matrix Factorization. Moreira et al. [22] also used search queries as context for the whole sequence. The effect of including page events in the sequence is only reported by [9], which can be considered to be most similar to our work. However, since the focus of their work is particularly on the challenge, the benefits of non-item information and different modeling approaches are not systematically explored and cannot be investigated completely from outside the challenge’s evaluation framework. Following this, [7] introduces the notion of non-items to be included in sequential recommender models, which is the work we directly build upon here. This modeling has since then also been adapted to detect mentions of entities in chat conversations [39].

In contrast to previous works, we verify our modeling approaches on several datasets and recommender architectures and systematically model various types of non-item pages in the sequence to consider the dependencies to items in chronological sequence order.

3 Methodology

In this section we discuss the problem setting, including our modeling approach for various types of non-item pages. We introduce the setting for HypTrails as well as the foundations of sequential recommender models and our adaptations.

3.1 Problem Setting

To formalize the problem setting, we closely follow [8] and introduce our notation as follows: Our work aims to solve the task of recommending items to a user given the sequence of previous items and non-items. We define the item set as $\mathcal{V} = \{v_1, v_2, \dots, v_{|\mathcal{V}|}\}$ and the set of non-items as $\mathcal{LP} = \{p_1, p_2, \dots, p_{|\mathcal{LP}|}\}$. An interaction i is either an item $v \in \mathcal{V}$ or a non-item page $p \in \mathcal{LP}$. The set of users is defined as $\mathcal{U} = \{u_1, u_2, \dots, u_{|\mathcal{U}|}\}$. For each user u , we can denote the sequence of interactions as $s_u = [i_1, i_2, \dots, i_n] \subseteq \{\mathcal{V} \cup \mathcal{LP}\}^n$. We use the term ‘*session*’ synonymously for a sequence of interactions. The set of all sequences is defined as $\mathcal{S} = \{s_{u_1}, s_{u_2}, \dots, s_{u_{|\mathcal{U}|}}\}$. Formally written, our goal is to solve the task of predicting the next item $i_{n+1} \in \mathcal{V}$ for every interaction i_n in each sequence $s_u \in \mathcal{S}$.

Additionally, we have an optional set $\hat{\mathcal{A}}$ of categorical attributes $\hat{a}$. These attributes can be, for example, tags like “category:shoes” or “genre:horror”. The set of all possible combinations of attributes that occur in the dataset is $\hat{\mathcal{C}} \subset 2^{\hat{\mathcal{A}}}$. An interaction i can be associated with one or multiple attributes $\hat{c}_i \in \hat{\mathcal{C}}$.

3.2 Non-Item Pages

In this section we describe the types of non-item pages that we aim to include in our models in more detail.

In almost any setting where sequential recommendation is usually applied, the users' navigation does not consist exclusively of items or item pages. For example, web-shops often have additional pages, some of which are not relevant for inferring the users' intent (e.g., payment information or legal notes), while others are strongly related to items and therefore to the users' intent. Examples of these pages include search results (where a user has either typed out a search query or selected a set of filters like color or size), category pages (e.g., "women shoes" or "men shirts"), or blog pages containing additional content provided by the shop's owner (e.g., pages comparing different types of hiking shoes regarding their suitability for travelling in mountains or other areas).

These pages often provide even clearer hints towards the user's intent ("I want to go hiking in the mountains") than could be inferred only from their item clicks. Similar types of non-item pages exist also in other settings: for example, streaming services often provide info pages for actors (listing all of the movies or shows that the actor stars in), category pages (listing all content for a specific genre like horror, drama, ...) or tags (e.g., "high suspense"). While these kinds of pages are often not included in public datasets, they usually exist within the respective companies and can be used to improve recommendations.

In this paper, we consider three types of non-item pages:

- (1) a list of items: e.g., a search result page
- (2) a single non-item page: e.g., a blog post with information about items
- (3) a list of non-item pages: e.g., a category page for all kinds of clothing, which lists category pages for pants, shoes etc.

All of these pages can be represented by different strategies, for example based on their content (the text in the case of blog posts and the contained items in the case of list pages) or separate unique ids.

3.3 Modeling Non-Item Pages

For each of the three types of non-item pages (lists of items, lists of non-item pages and other non-item pages) we explore three options for representing the page: using or generating a unique id, generating a content-based id for the page, or generating an embedding for the page's content.

Unique Page ID (UPID). The simplest solution is to assign a unique id to each non-item page p_n . This is possible for any kind of page, for example by employing the URL used to access the page, the title of the page or - as a last resort - by assigning a random id to each unique page. However, this comes with several drawbacks: First, there are instances where the generated unique id is not informative. This is for example the case if a URL is generic and content is generated on the fly, so the relevant information is lost. On the other hand, if the URL is too specific (e.g. because it contains a search query), the associated id may be too rare for a model to extract useful information. The vocabulary size will increase and become unmanageable, and the models will learn to predict non-items, even though the goal is item recommendation. While predictions can be filtered afterwards, valuable capacity of the model will be used for a task irrelevant to next-item prediction. Nevertheless, we include UPID as a baseline.

Content-based Page ID (CPID). To tackle these challenges, we utilize the content of non-item pages. We can use categorical attributes to create meaningful and shared ids for similar non-item pages. In the easiest case, a non-item page $p_n \in \mathcal{LP}$ already has some categorical attributes $\hat{c}_{p_n} = \{\hat{a}_1, \hat{a}_2, \dots, \hat{a}_{|\hat{c}_{p_n}|}\}$, $\hat{a} \in \hat{\mathcal{A}}$, assigned, for example because

it is a category list page (all items from category “shoes”) or because it has been manually tagged. For list pages we can also construct page attributes from the attributes of the items contained in the list. Therefore the set of non-items becomes $\mathcal{LP}_{\text{CPID}} = \{\hat{c}_p \mid p \in \mathcal{LP}\}$. For non-list pages, this strategy is not applicable if there are no attributes assigned directly to the page.

Page Embedding (PE). By building ids from attributes, as proposed in the previous paragraph, our vocabulary still grows by size $|\mathcal{LP}_{\text{CPID}}|$. Therefore, we propose to use one single placeholder id to represent all non item pages and encode the content of a page with an embedding, mitigating the issue of increasing the vocabulary size. This also allows the use of any vector representation for the page. This representation $\hat{p}_n \in \mathbb{R}^m$ (m : dimension of the embedding vector) can be extracted from the categorical attributes of a page p_n , from textual content, images, or a search query or be given directly. In general, this embedding representation can be applied to any kind of non-item page.

3.4 Verifying the Influence of Non-Item Pages using HypTrails

Following our intuition, it is reasonable to assume that non-item have an influence over the users’ navigation behavior, as argued in Section 1. Verifying this influence in a statistical way would provide a clear signal that incorporating non-items into a sequential recommender should enable the model to better understand the users’ intent. To achieve this, we can formulate different hypotheses about the users’ navigation behavior and compare them to see which one fits our data best. In our case, we are mostly interested in two hypotheses, where one assumes that non-items pages are influential and the other that they are not. Assuming that non-item pages are influential, we construct a ‘structural’ hypothesis, which expresses the belief that users only move from a non-item page to items of the page’s category. The opposite hypothesis (‘uniform’) is that there is no influence, and that users move uniformly between pages of the shop. A suitable method to compare these hypotheses is the HypTrails [25] framework. It was developed to compare hypotheses about navigation trails of users on the web. A hypothesis in this framework describes a belief about the transition behavior between states, in our case between the pages of a web shop. HypTrails enables us, given several such hypotheses, to rank them according to how well they explain the observed data. If the ‘structural’ hypothesis is ranked higher than the uniform hypothesis, we can conclude that non-item pages indeed contain valuable information about the following user interactions.

3.4.1 Background. The HypTrails approach utilizes Bayesian inference to compute the likelihood of the observed data D , in our case the interaction sequences $s \in S$, under the assumption of a hypothesis H . For data D , items and non-items are modeled as *states* and the navigation behavior is represented by a first-order Markov chain in form of a *transition* matrix, containing the likelihood of moving from one state to another. For a hypothesis H , we manually construct a pseudo transition matrix, representing our belief about the likelihood of transitions. For example, according to the ‘uniform’ hypothesis, all transitions are equally likely (see Equation (2)); or for the ‘structural’ hypothesis, the likelihood of going from a non-item page to all pages of the same genre is equal, while the likelihood to go to pages of different genres is zero (see Equation (3)).

HypTrails can then be used to automatically construct a Bayesian Dirichlet prior from these hypotheses. The Dirichlet parameters α are directly influenced by a concentration factor k , which represents the strength of our belief in a hypothesis.

We can now compute the marginal likelihood $P(D \mid H)$ as evidence for a hypothesis given different concentration factors k . Comparing the evidence between multiple hypotheses yields a ranking: The hypothesis with the highest

evidence provides the best explanation for the data. This allows us to compare hypotheses for a range of beliefs, from “allowing some errors” to “fitting accurately”. For a higher concentration factor k the difference in evidence between two hypotheses becomes more pronounced if one hypothesis fits the data better.

To rank the hypotheses, we plot their evidence for different values of k in a line plot. If the lines do not cross, it is evident that one hypothesis fits better than the other.

Formally, the evidence for a hypothesis H is calculated as follows:

$$P(D | H) = \prod_i \frac{\Gamma(\sum_j \alpha_{ij}) \prod_j \Gamma(\alpha_{ij} + t_{ij})}{\prod_j \Gamma(\alpha_{ij}) \Gamma(\sum_j (\alpha_{ij} + t_{ij}))} \quad (1)$$

Here, t represents the actual transition counts in our data D and α are Dirichlet parameters that incorporate the prior, corresponding to a specific hypothesis H (cf. Section 3.4.2). The resulting evidence scores can now be used to compare multiple hypotheses with each other and rank them with respect to the concentration factor k . Note that the evidence values of a hypothesis can not be interpreted in isolation, but can only be used for comparison with another hypothesis. For further details, we refer the reader to Singer et. al [25].

3.4.2 Approach. To use HypTrails, we need to define states, transitions, and hypotheses. In our case, items and non-items will be mapped to states and transitions are obtained from the sessions in our dataset.

States. As states in HypTrails are discrete and we need some knowledge about the states to formulate meaningful hypotheses, we employ the previously introduced Content-Based Page ID (CPID) as the only way to map non-item pages to informative states. To represent items as states, we follow the idea of network abstraction from [17] and aggregate all items of the same attribute combination into one single state. This allows us to better represent the more abstract hypothesis that a non-item page is followed by any item of the same category and also reduces computation time significantly. We write this set of aggregated items with content-based ids as $\mathcal{V}_{\text{CID}} = \{\hat{c}_v \mid v \in \mathcal{V}\}$. Now, we can define the states $\mathcal{K}$ as the disjoint union of both the aggregated items and non-items represented by their combination of categories $\hat{c}$:

$$\mathcal{K} = \mathcal{V}_{\text{CID}} \sqcup \mathcal{LP}_{\text{CPID}}$$

Transitions. First, we gather the multi-sets $\mathcal{M}_{s_u}$ of all transitions in a session $s_u = [i_1, i_2, \dots, i_n]$ mapped to their respective Content-based ID $\hat{i} \in \mathcal{K}$ as $\mathcal{M}_{s_u} = \{(\hat{i}_t, \hat{i}_{t+1}) \mid t \in [0, n)\}$. We can now construct the multi-set of all transitions in the dataset:

$$\mathcal{M} = \bigcup_{s_u \in S} \mathcal{M}_{s_u}.$$

Hypotheses. For each of our hypotheses, we define a matrix $H \in [0, 1] \subset \mathbb{R}^{(|\mathcal{K}|)^2}$ which contains the transition probabilities between our states, which are both items from $\mathcal{V}_{\text{CID}}$ and non-item pages from $\mathcal{LP}_{\text{CPID}}$.

We describe and formalize four hypotheses as follows:

- **uniform:** Non-Item pages have no special influence on the following items, therefore all transition probabilities are distributed uniformly. Therefore we set:

$$H_{x,y}^{\text{uni}} = \frac{1}{|\mathcal{K}|} \quad (2)$$

- **structural:** Non-Item pages directly influence the following interaction - we assign equal transition probabilities from each non-item page to the corresponding items of the same category $\hat{a}$ and zero probability to items of other categories and other non-item pages as follows:

$$H_{x,y}^{\text{struct}} = \begin{cases} \frac{1}{|\mathcal{K}|} & , \text{if } x \in \mathcal{V}_{\text{CID}} \\ 1 & , \text{if } x \in \mathcal{LP}_{\text{CPID}}, y \in \mathcal{V}_{\text{CID}} \text{ and } \hat{a}_x = \hat{a}_y \\ 0 & , \text{otherwise} \end{cases} \quad (3)$$

- **uniform+structural:** A mixture of both previous hypotheses: non-item pages have a higher transition probability to items of the same category than to other pages and we set the transition probabilities to the normalized sum of the two previous hypotheses:

$$H_{x,y}^{\text{uni+struct}} = \begin{cases} \frac{1}{|\mathcal{K}|} & , \text{if } x \in \mathcal{V}_{\text{CID}} \\ \frac{2}{1+|\mathcal{K}|} & , \text{if } x \in \mathcal{LP}_{\text{CPID}}, y \in \mathcal{V}_{\text{CID}} \text{ and } \hat{a}_x = \hat{a}_y \\ \frac{1}{1+|\mathcal{K}|} & , \text{otherwise} \end{cases} \quad (4)$$

- **data:** The transition probabilities actually found in the data, serving as an upper bound. We count transitions from state x to state y , normalizing by the count of any transition from x and assign the following probabilities:

$$H_{x,y}^{\text{data}} = \frac{|\{(x,y)\} \cap \mathcal{M}|}{\sum_{y' \in \mathcal{K}} |\{(x,y')\} \cap \mathcal{M}|} \quad (5)$$

3.5 Adapting Sequential Recommender Models for Non-Item Representations

In general, most sequential recommender models follow a similar setup to the one shown in Fig. 2a. The input of the model is a sequence of item ids, which are fed to an *embedding layer* for learning the representation of the sequence of interactions. This layer includes at least an embedding of the item ids and an optional positional embedding. The embedded input is then forwarded to the actual architecture of the model, which can be seen as a black box of *sequential recommender layers* here. The output of these layers is mapped back to the item space through a *projection layer*. Oftentimes, the original item embedding from the first layer is reused here.

As we want to study the general impact of including non-item pages, we include a variety of sequential recommender models fitting this setup in our experiments. We utilize RecBole [41], a popular recommendation framework, which already includes a number of sequential recommendation models used for next-item prediction. We further limit our selection to models without user or item features. This leaves us with the following models, many of them popular baselines:

- GRU4Rec[28] is a popular baseline, which utilizes multiple layers of Gated Recurrent Units for sequential recommendation.
- CASER [29] is a convolution-based network, which employs both horizontal and vertical convolution layers to model sequences as high-order Markov Chains.
- NARM [18] enhances recommendations by combining recurrent neural networks with an attention mechanism.
- NextITNet [38] analyzes and addresses some limitations of CASER by stacking dilated 1D-convolutional layers to improve representations of both short- and long-range item dependencies.
- SASRec [16] were the first to introduce a model based on self-attention for sequential recommendation.

- BERT4Rec [26] adopts the famous BERT [5] setup for recommendation by using bi-directional transformer layers and the cloze task [30], which masks items randomly for training.
- CORE [13] introduces a representation-consistent encoder, which couples the representation space for both the encoding and decoding by using a linear combination of the item embeddings as session embedding. The items themselves are also encoded with self-attention blocks.
- LightSANs [6] also uses self attention, but adapts a low-rank decomposed self-attention to capture latent factors in the interactions and a decoupled positional encoding in addition.

All these models can be used to include non-items with unique ids directly. To adapt them to different types of non-item representations, we follow the approach presented in [7] for BERT4Rec and SASRec and their variants for including non-item pages called P-SASRec^c and P-Bert4Rec. The embedding layer in the basic form for all models contains

- (i) an embedding $E_V \in \mathbb{R}^{|\mathcal{V}| \times d}$ of the identifier and
- (ii) (optionally) an embedding $E_G \in \mathbb{R}^{N \times d}$ to encode the position of the items in the sequence, with N as the maximum input sequence length.

When we create unique ids for each page p , the embedding of the id will be $E_V \in \mathbb{R}^{|\mathcal{V} \cup \mathcal{L}^P| \times d}$ instead of $E_V \in \mathbb{R}^{|\mathcal{V} \cup \mathcal{L}^P_{CPD}| \times d}$ if the ids are based on the categorical attributes.

For including the page information as categorical attributes, the embedding is defined as $E_V \in \mathbb{R}^{(|\mathcal{V}|+1) \times d}$ instead, as we add only one general token for all pages p . To include the categorical attributes of the items directly, we therefore add

- (iii) a multi-hot encoding of the attributes $E_{\mathcal{A}} \in \{0, 1\}^{|\mathcal{A}|}$ and a linear layer for resizing $E_{\mathcal{A}}$ to the hidden size d : $l_{\mathcal{A}}(x) = Wx + b$ with the weight matrix $W \in \mathbb{R}^{|\mathcal{A}| \times d}$ and bias $b \in \mathbb{R}^d$

to our embedding layer. Further, we adapt the models to utilize arbitrary non-item representations as follows: Assuming that there is a latent representation of size R for i called $\hat{r}_i \in \mathbb{R}^{|\mathcal{R}|}$, we add

- (iv) one linear layer $l_R(x) = Wx + b$ with the weight matrix $W \in \mathbb{R}^{R \times d}$ and bias $b \in \mathbb{R}^d$ to scale the latent representation to the hidden size d .

To create the final output of the embedding layer, all layers are summed to include the additional outputs of $l_{\mathcal{A}}$ and l_R . For each timestep t , we use the id embedding $e_t = E_V(i_t)$ of interaction i_t , the positional embedding $g_t = E_G(t)$, the keyword embedding $a_t = l_{\mathcal{A}}(E_{\mathcal{A}}(\hat{a}_{p_t}))$ and the scaled latent representation $r_t = l_R(\hat{r}_{p_t})$ to compute the sum $e_t + a_t + r_t + g_t = h_t^0$ as summarized in Figure 2b, which is then used as the input to the model. The other layers of the models remain unchanged. For the masked training of BERT4Rec, we make sure to also mask attributes and representations whenever an interaction is masked. Finally, training and calculation of the loss remain the same as in the original models, without differentiation between non-items and items, i.e., non-items are also masked in BERT4Rec. However, we do ensure in our data preprocessing that all sequences end with an actual item.

4 Datasets

This section introduces the datasets used in our experiments. The statistics for all datasets can be found in Table 1. We generally follow the data processing steps from [7]. We filter sessions to end with an actual item, but in contrast to [7] we include this step in our preprocessing instead of the dataloader leading to slightly different dataset statistics.

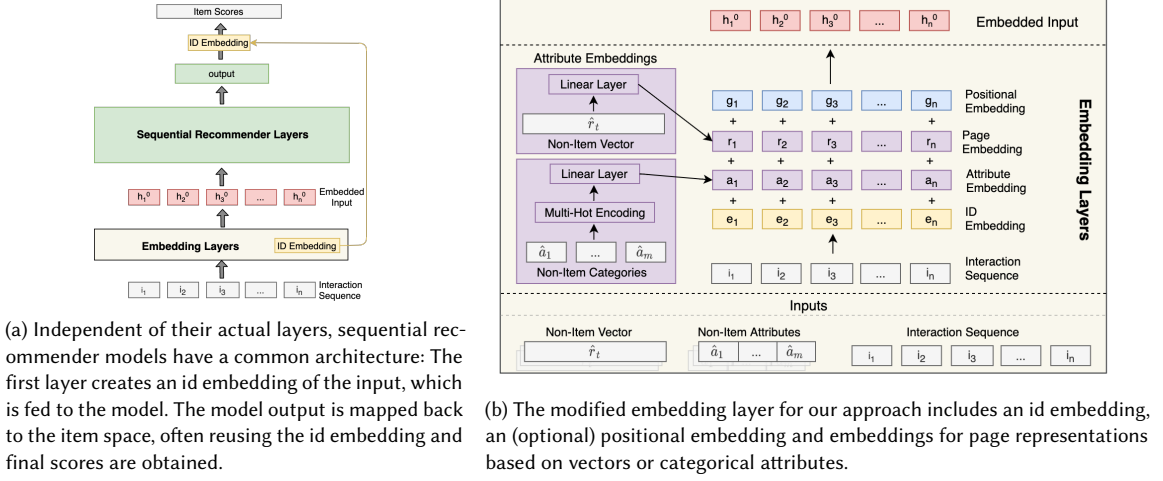


Fig. 2. The common setup for sequential recommendation models and our approach for including non-item representations.

4.1 Synthetic Dataset SynDS

To assess the effectiveness of our proposed approaches in enabling sequential recommender models to learn from non-item pages, it is crucial to use a dataset that ensures that the non-item pages contain valuable information. Therefore, we create a synthetic dataset SynDS, based on the ML-20m dataset. The ML-20m¹ dataset consists of movie ratings from users and is a popular benchmark for sequential recommendation. For each movie there is metadata in the form of genres available. We use the ratings as interactions and build interaction sequences, as for example in [26]: For each user we construct a sequence of movie interactions by using each rated movie as an item and sorting them by the timestamp of the rating. We also remove users and items with less than four ratings. We split the data randomly by users, with 80% of users for training, 10% for validation and 10% for testing, utilizing the exact same split as in [7] for comparability (cf. Section 6.4).

As the dataset does not contain any non-item interaction, we create three dataset variants, containing different levels of information based on the movie genres. Note that the results obtained on these datasets are not comparable to the original ML-20m, as we leak information about the following movie. The datasets are designed to demonstrate how models can leverage information from additional non-item pages in a controlled setting, where we ensure that these pages contain valuable information, or in the case of Random-SynDS confirm that the models are not negatively impacted by the introduction of noise.

For each dataset, we represent non-item pages either by building a Content-based Page ID (CPID) with the genre combination as the id or by a page embedding with the genres as embedded categorical keywords.

4.1.1 Prev-SynDS. In our first experiment, we want to explore the benefit of highly informative non-item pages. Therefore, for each movie interaction in a sequence, we add a non-item page tagged with the movie's genres immediately prior to that movie. This simulates a setting in which the user first navigates to a category page listing all movies of a particular genre and selects a specific movie afterwards. Each added non-item page contains highly relevant information for the recommendation task, as the genre for the next movie is now already known. For example, instead of the movie

¹<https://grouplens.org/datasets/movielens/20m/>

sequence “Alien → Jaws → Shrek” the enriched sequence would be “Horror|Sci-Fi → Alien → Action|Horror → Jaws → Adventure|Children → Shrek”. We show the distribution of the genres within the complete dataset in Figure 3a. The distribution is highly skewed, with Drama and Comedy covering more than 35 % of the interactions. While the popular genres might not be very informative on their own, a non-item page has ≈ 2.66 genres on average and therefore the genre combinations are far more distinctive.

4.1.2 Group-SynDS. As Prev-SynDS contains a very strong prior, we also explore a more realistic setting by only adding a non-item-page when the genre of the following movie is from different genres than the movies before. For example, instead of “Horror|Sci-Fi → The Thing → Horror|Sci-Fi → Alien → Action|Horror → Jaws” as in Prev-SynDS, we generate “Horror|Sci-Fi → The Thing → Alien → Action|Horror → Jaws”. This reduces the number of non-item interactions to about 19 million (cf. Table 1).

4.1.3 Random-(X)-SynDS. To cover the case of non-item pages containing very noisy signals or no useful information at all, we generate a set of datasets with randomized non-item pages to explore the susceptibility of the models to noise potentially introduced by adding these pages. We therefore add a non-item page before each movie interaction as in Prev-SynDS and shuffle a given percentage of the pages including their genres. We denote the ratio in the name, e.g. Random-0.1-SynDS contains 10% non-item pages with randomized genres. Please note that some of them will still be assigned correct genres by chance, so the actual number of ‘wrong’ genres might be lower. For the example from Prev-SynDS, the sequence could look like “Musical|Children → Alien → Comedy|Adventure → Jaws → Documentary → Shrek” in Random-1.0-SynDS.

4.2 Coveo

Published for the SIGIR 21 challenge, the Coveo² dataset [27] is an extensive e-commerce dataset that captures browsing and search interactions. The captured browsing behavior consists of interactions with a product page or the view of a web page. A view can be related to a product, but is often only represented by its hashed URL. The dataset also provides search requests, including the embedded search query as a vector of length 45 and the list of retrieved items.

Moreover, the dataset includes metadata such as hierarchical categories, price, and preprocessed text and image embeddings for products. As our focus in this paper is on incorporating non-item pages, we are not directly utilizing them. However, we use the categories of items to build representations for the lists of retrieved items from search requests. For preprocessing, we first drop all duplicate interactions (which are erroneous artifacts of the dataset), keeping only the first one. Additionally, a product view often creates a page view event linked to the product with the same timestamp. To condense these interactions, we drop the page view event, which is less informative, and only keep the corresponding product view. We ensure that each session ends with an actual item by removing page views from the tail of each session until the last interaction is a product view. We then drop sessions which contain less than two interactions. Furthermore, items and page views that occur less than five times in the dataset are dropped from the sequences.

As we want to examine both types of non-item pages in the dataset separately, we split the data into two sets: For *Coveo-Pageview*, we exclusively use item interactions and page views as non-item pages. As there is no further information available, we can only represent a page view with separate ids, namely their hashed URL. For *Coveo-Search*, we reduce the dataset to sessions containing at least one search event. For each search event, we use the list of retrieved

²<https://www.coveo.com/en/ailabs/sigir-ecom-data-challenge>

Table 1. Statistics of the preprocessed datasets SynDS, Coveo-Search, Coveo-Pageview and Fashion, showing the number of users, items and unique pages, page attributes and their permutations as CPIDs as well as the number of interactions with items and pages and the average session lengths. Coveo-Search attributes are based on the first or the most frequent categories, Fashion attributes on categories and filters.

Name	Dataset: Variant	Users:	Items:	UPID:	Attributes:	CPID:	Interactions:		Avg. session length:	
		$ U $	$ V $	$ \mathcal{LP} $	$ \hat{\mathcal{A}} $	$ \mathcal{LP}_{CPID} $	$ i_V $	$ i_{LP} $	i_V	with i_{LP}
SynDS	Previous/Random Group	138 493	26 744	-	20	1 329	20 m	20 m	144.4	288.8
		138 493	26 744	-	20	1 329	20 m	19 m	144.4	281.9
Coveo	PageViews Search	1 318 922	29 268	73 217	-	-	8.6 m	6.6 m	6.6	10.3
		134 881	27 013	-	112/112	166/166	895 760	185 259	6.6	8.0
Fashion		199 474	8 412	-	204/5 647	7 207/27 863	4.3 m	1.3 m	21.7	28.2

items as a non-item page. We create several representations for search pages: First we compute the three most frequent categories of the products in the retrieved list. As a second categorical representation, we use the categories of the first product in the search result list. We show the distribution of the categories (based on the first search result) in Figure 3b. Both can now be represented by a category-based id or by embedding the categories. We also use the product id of the first search result directly as a representation, therefore using an item to represent the non-item page and not increasing the vocabulary at all. As all these representations depend on the quality of the retrieved item list, we also use the search query embedding vector, which is a direct expression of a user’s interest. In comparison to the genres in SynDS, the distribution seems to be even more skewed. This could be explained by the categories being hierarchical - the limited number of ‘root’ categories will be present in every search page, categories in the ‘mid level’ far less, and the finest granularity of categories will be even rarer.

We split both Coveo-Pageview and Coveo-Search by time. We sort all item and non-item interactions by time and divide them at two specific points in time, such that about 70% of interactions are in the first split. These are grouped by user to build sequences for training. The second split covers about 15% of interactions and is used for validation, while the remaining interactions are for used testing, both again grouped by user to build sequences.

4.3 Fashion

The Fashion dataset is composed of interactions with product and list pages from a major online fashion store, collected over 20 days. A product page shows a single item, while a list page displays multiple items of particular (sub-)categories with additional filters applied by the user. Categories and filters are available as attributes for each page, and while the information is partially overlapping, filters usually contain more detailed information.

For example, a list page with the category “shoes” can have more specific filters, such as “type:shoes”, “color:black” and “material:leather”. Multiple interactions with the same product or page in a row are condensed into a single interaction. Furthermore, we ensure a minimum sequence length of 3, a maximum of 200 and ensure an item occurs at least five times in the data as in [26].

We sort all item and non-item interactions by time and divide them at two specific points in time, such that about 70% of interactions are in the first split. These are grouped by user to build sequences for training. The second split covers about 15% of interactions and is used for validation, while the remaining interactions are for used testing, both again grouped by user to build sequences.

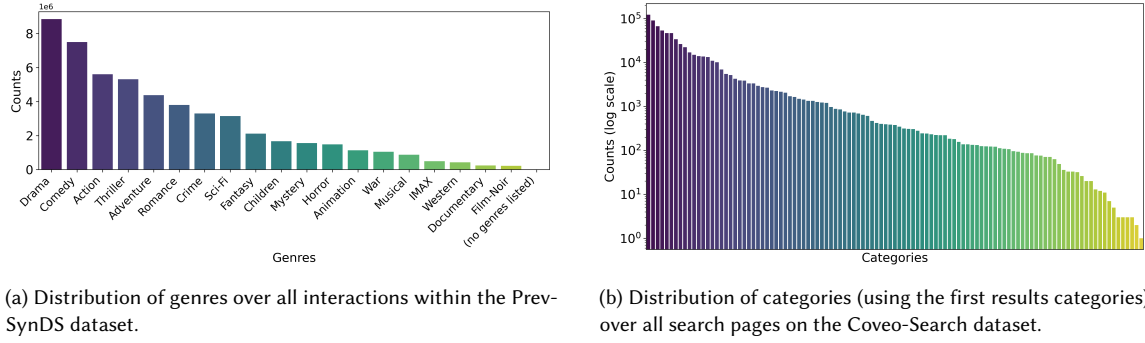


Fig. 3. Distribution of attributes within SynDS and Coveo-Search dataset.

5 Experiments

In this section, we introduce the research questions we aim to answer and an explanation of the experimental setup, present the conducted experiments, and subsequently discuss our results.

In our experiments, we aim to address the following five research questions:

- RQ1 Do non-item pages have an influence on the following item interaction?
- RQ2 Can sequential recommender models utilize non-item pages for predicting the next item?
- RQ3 Are there gains from including non-item pages even without access to their content?
- RQ4 What representations are useful to integrate search requests as non-item interactions?
- RQ5 Can we find good representations to integrate list pages as non-item pages?

We aim to answer these questions in our following experiments and provide the common experimental setups next:

5.1 Experimental Setup

We train all our models with the Adam optimizer and cross-entropy loss. In line with previous research [7], we use the same hyperparameter settings for all models and keep the embedding and inner size, as well as the number of layers the same for all models. We use the following hyperparameter settings, following [7] where applicable: The hidden or embedding size for all models is 64 and the inner size 256. For all transformer-based models and GRU4Rec, we set the number of layers (and heads) to 2 and the dropout to 0.2. For BERT4Rec the mask ratio is 0.2 and the finetuning probability 0.1.

For CASER and NextItNet, we base the additional hyperparameters on the RecBole defaults: we employ a dropout rate of 0.4, a Markov chain length of 5 and 4 vertical and 8 horizontal filters for CASER. For NextItNet this means a kernel size of 3, a width of the convolutional filter of 5 and dilations of [1, 4].

As SynDS is based on the MovieLens data, we also use this dataset's established sequence length of 200 (cf. [26]). For the remaining datasets, we aim to include at least 95% of sessions in their entirety while keeping the sequence length reasonable for efficiency and choose the sequence lengths accordingly, rounding up. The number of epochs was determined based on preliminary experiments, where the loss curve showed convergence with no further meaningful improvements.

Overall, we do not aim to find the overall optimal model, but to systematically investigate the utility of non-item pages and the effect of their inclusion in several different models. Therefore, we do not conduct hyperparameter studies, but

rely on the parameter settings described above, which are anchored in previous research.

We train all models on the seeds [212, 6, 10, 404, 42] and report the average and standard deviation over the five runs, except our experiments on Random-X-SynDS and Fashion due to limited computational resources, for which we report only the results on seed 212. We keep our batch size consistent for all experiments and choose 64 to utilize our GPUs best.

Evaluation Settings. For each sequence in the test (and validation) data, we use the last item as the prediction target and the rest of the test (or validation) sequence as input, following previous studies [10, 38].

Note that we have already ensured in our data preprocessing that the target is an item and not a non-item. To evaluate the quality of recommendation for actual items, we set the probability of all non-items in the model output to 0 before computing the metrics. We report the full *Hit Rate (HR)* and the *Normalized Discounted Cumulative Gain (NDCG)* at $k \in \{1, 5, 10\}$ without any sampling. Additionally, we use the dependent Student’s t-test for paired samples to calculate p -values on user-level for significance testing and report significant increases over the baselines for $p < 0.01$ and $p < 0.05$ for our main experiments.

We provide the code and configurations for our experiments in our repository ³.

5.2 RQ1: Do Non-Item Pages have an influence on the following item interaction?

Our first goal is to verify our hypothesis about the importance of non-item pages for following items within click sequences. We utilize the HypTrails framework (cf. Section 3.4) on four of our datasets to test whether a non-item page with a specific category is generally followed by items of the same category. We use SynDS and Random-X-SynDS with genre-based CPIDs and Coveo-Search and Fashion with category-based CPIDs.

In Figure 4 we plot the evidence or marginal likelihood against the concentration factor k for our datasets and compare four hypotheses against each other: The ‘structural’ hypothesis (non-items influence following interactions), the ‘uniform’ hypothesis (non-items have no special influence on following interactions), the ‘uniform-structural’ hypothesis (non-items have a small influence on following interactions) and the ‘data’ hypothesis, which serves as an upper bound as it fits the data perfectly.

A higher evidence shows that a hypothesis provides a better explanation for the observed data. Recall from Section 3.4 that the k -factor represents the amount of belief in a hypothesis or how strictly deviations from the actual data are penalized. For $k = 0$ we do not believe in a hypothesis at all and allow any amount of deviations from the hypothesis in the data. Therefore all hypotheses yield the same evidence in our plots. Once we increase k and become more strict in our belief, we see greater differences in evidence between the hypotheses, allowing us to rank them.

On the Prev-SynDS (see Figure 4a) the structural hypothesis ranks second-best, as it can explain most of the observed transitions. At higher concentration factor k the evidence drops, as it assumes uniformly distributed probabilities from items to other states.

Only the data hypothesis performs better, as it exactly fits the transition probabilities. Overall, the uniform hypothesis has a lower evidence compared to the structural hypothesis. The combined hypothesis ranks slightly higher, although only by a small difference. In comparison, the structural hypothesis gives the best explanation for the transitions in our dataset (except for the upper bound data hypothesis), which verifies that our dataset indeed contains influential non-item pages. On Random-1.0-SynDS, we see that the signal of the non-item pages has been diluted as intended. For a lower concentration factor, the differences between the three hypotheses are minimal, while for higher values, the

³<https://github.com/LSX-UniWue/non-items-recbole>

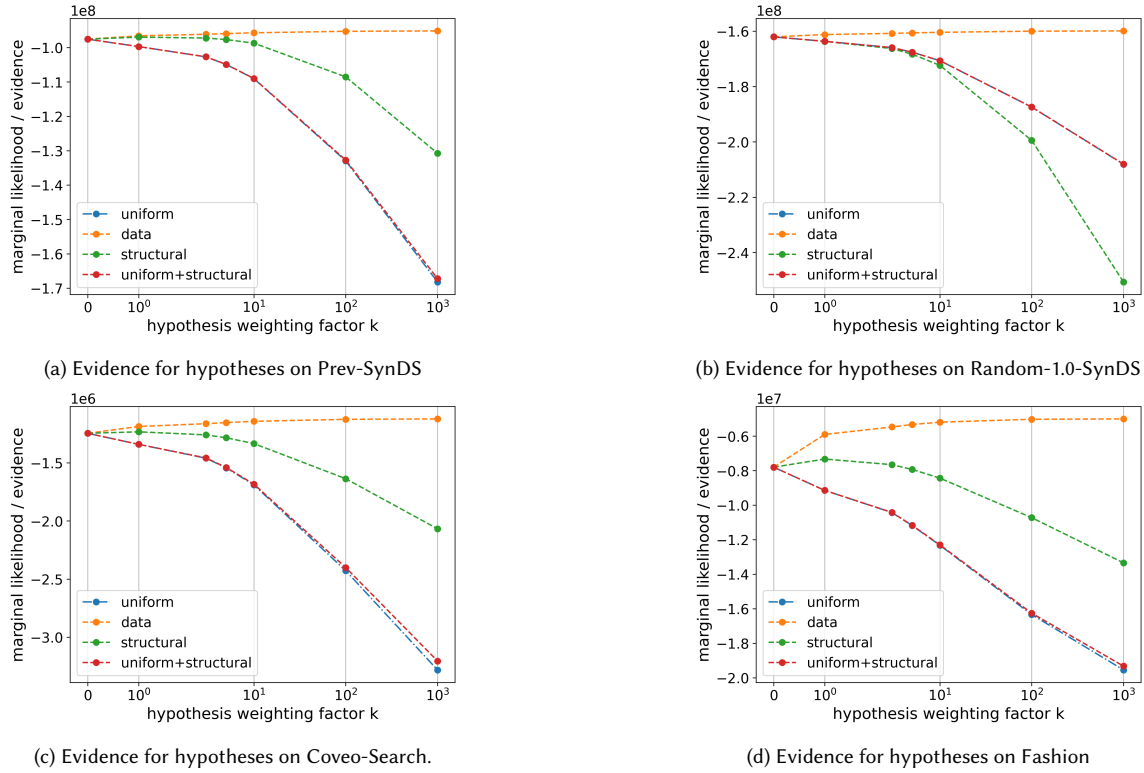


Fig. 4. The evidence or marginal likelihood for our hypotheses on SynDS, Coveo-Search and Fashion for different hypothesis weighting factor k . A higher evidence means a hypothesis is a better explanation for the data, while k can be interpreted as how strongly you believe in the hypothesis.

structural hypothesis shows a harsher drop. For both Coveo-Search and Fashion we can also verify the meaningfulness of non-item pages, similar to Prev-SynDS. The evidence for the structural hypothesis is constantly above the uniform hypothesis. The combined hypothesis is also slightly better than the uniform hypothesis for both datasets, as the structural part of the hypothesis includes at least some explanation about the transitions.

One limitation of HypTrails in our setting is the need for discrete states, which means that we can only represent non-item pages by ids and not their content. Additionally, we need some knowledge about the non-item pages to build meaningful hypotheses. Therefore, we can neither apply HypTrails for Coveo-Pageview, as we have no information about the non-item pages there, nor to non-item representations based on, for example, queries. A second restriction is that only first-order dependencies are modeled, but we would expect non-item pages to have influence on more than the single next interaction. Nevertheless, our results already show a positive influence for first-order dependencies, so we estimate the actual influence to be even higher. Overall, our results give us the necessary evidence to consider non-item pages as a valuable information source for our following experiments.

Table 2. Average Hitrate and NDCG with standard deviation over five random seeds for non-item models on the SynDS dataset. This includes the genre-specific popularity baseline (Genre-POP) and the baseline model on item interactions only (Items-BL). We report results for both Prev- and Group-SynDS datasets for including Non-Item Pages with both page embeddings (PE) and id embeddings (ID). Results marked with * show a significantly higher performance compared to the items-only baseline for $p < 0.01$.

Model	Metrics	Genre-POP	Items-BL	Prev-ID	Prev-PE	Group-ID	Group-PE
BERT4Rec	HR@1	.010	.010 $\pm$.001	.119* $\pm$.012	.040* $\pm$.048	.294* $\pm$.023	.308* $\pm$.012
	HR@5	.031	.042 $\pm$.003	.241* $\pm$.025	.087* $\pm$.092	.534* $\pm$.043	.559* $\pm$.009
	HR@10	.050	.075 $\pm$.004	.298* $\pm$.031	.111* $\pm$.108	.624* $\pm$.046	.653* $\pm$.007
	NDCG@5	.020	.026 $\pm$.002	.183* $\pm$.019	.064* $\pm$.071	.421* $\pm$.034	.441* $\pm$.009
	NDCG@10	.026	.037 $\pm$.002	.201* $\pm$.020	.072* $\pm$.076	.450* $\pm$.035	.471* $\pm$.009
Caser	HR@1	.010	.018 $\pm$.001	.319* $\pm$.013	.301* $\pm$.011	.322* $\pm$.011	.323* $\pm$.010
	HR@5	.031	.054 $\pm$.002	.576* $\pm$.012	.557* $\pm$.011	.570* $\pm$.012	.573* $\pm$.009
	HR@10	.050	.084 $\pm$.005	.674* $\pm$.006	.658* $\pm$.007	.666* $\pm$.011	.671* $\pm$.008
	NDCG@5	.020	.036 $\pm$.001	.455* $\pm$.012	.436* $\pm$.011	.453* $\pm$.011	.456* $\pm$.009
	NDCG@10	.026	.046 $\pm$.002	.487* $\pm$.010	.469* $\pm$.010	.485* $\pm$.011	.487* $\pm$.009
Core	HR@1	.010	.015 $\pm$.001	.350* $\pm$.002	.303* $\pm$.007	.348* $\pm$.003	.287* $\pm$.003
	HR@5	.031	.059 $\pm$.002	.604* $\pm$.002	.547* $\pm$.007	.602* $\pm$.002	.529* $\pm$.006
	HR@10	.050	.100 $\pm$.003	.699* $\pm$.002	.642* $\pm$.006	.695* $\pm$.001	.627* $\pm$.007
	NDCG@5	.020	.036 $\pm$.001	.486* $\pm$.001	.432* $\pm$.006	.483* $\pm$.003	.415* $\pm$.004
	NDCG@10	.026	.050 $\pm$.002	.516* $\pm$.001	.463* $\pm$.006	.513* $\pm$.002	.447* $\pm$.004
GRU4Rec	HR@1	.010	.022 $\pm$.001	.355* $\pm$.002	.364* $\pm$.006	.363* $\pm$.003	.368* $\pm$.001
	HR@5	.031	.070 $\pm$.002	.612* $\pm$.002	.623* $\pm$.002	.617* $\pm$.003	.625* $\pm$.002
	HR@10	.050	.109 $\pm$.003	.705* $\pm$.003	.716* $\pm$.004	.706* $\pm$.003	.715* $\pm$.002
	NDCG@5	.020	.046 $\pm$.002	.492* $\pm$.001	.502* $\pm$.004	.498* $\pm$.003	.505* $\pm$.002
	NDCG@10	.026	.059 $\pm$.002	.523* $\pm$.002	.532* $\pm$.004	.527* $\pm$.003	.534* $\pm$.001
LightSANS	HR@1	.010	.026 $\pm$.001	.343* $\pm$.002	.345* $\pm$.003	.347* $\pm$.004	.347* $\pm$.002
	HR@5	.031	.078 $\pm$.001	.607* $\pm$.002	.609* $\pm$.003	.603* $\pm$.002	.603* $\pm$.002
	HR@10	.050	.118 $\pm$.001	.701* $\pm$.002	.703* $\pm$.002	.695* $\pm$.002	.694* $\pm$.002
	NDCG@5	.020	.052 $\pm$.001	.483* $\pm$.002	.485* $\pm$.003	.483* $\pm$.001	.483* $\pm$.001
	NDCG@10	.026	.065 $\pm$.001	.514* $\pm$.001	.516* $\pm$.003	.513* $\pm$.001	.513* $\pm$.001
NARM	HR@1	.010	.022 $\pm$.001	.355* $\pm$.004	.357* $\pm$.004	.356* $\pm$.003	.358* $\pm$.005
	HR@5	.031	.071 $\pm$.001	.617* $\pm$.002	.615* $\pm$.005	.613* $\pm$.002	.617* $\pm$.003
	HR@10	.050	.111 $\pm$.001	.711* $\pm$.002	.708* $\pm$.002	.705* $\pm$.003	.708* $\pm$.004
	NDCG@5	.020	.046 $\pm$.001	.495* $\pm$.003	.495* $\pm$.004	.493* $\pm$.002	.496* $\pm$.003
	NDCG@10	.026	.059 $\pm$.001	.525* $\pm$.003	.525* $\pm$.003	.523* $\pm$.002	.526* $\pm$.003
NextItNet	HR@1	.010	.007 $\pm$.000	.253* $\pm$.002	.209* $\pm$.112	.238* $\pm$.004	.192* $\pm$.103
	HR@5	.031	.022 $\pm$.001	.471* $\pm$.003	.388* $\pm$.199	.459* $\pm$.002	.375* $\pm$.192
	HR@10	.050	.036 $\pm$.001	.559* $\pm$.005	.463* $\pm$.228	.551* $\pm$.002	.455* $\pm$.224
	NDCG@5	.020	.014 $\pm$.001	.368* $\pm$.002	.304* $\pm$.158	.355* $\pm$.003	.289* $\pm$.151
	NDCG@10	.026	.019 $\pm$.001	.397* $\pm$.003	.328* $\pm$.167	.384* $\pm$.002	.314* $\pm$.161
SASRec	HR@1	.010	.025 $\pm$.002	.370* $\pm$.003	.368* $\pm$.002	.371* $\pm$.002	.361* $\pm$.003
	HR@5	.031	.081 $\pm$.002	.634* $\pm$.002	.630* $\pm$.003	.629* $\pm$.002	.618* $\pm$.003
	HR@10	.050	.123 $\pm$.001	.725* $\pm$.002	.721* $\pm$.002	.717* $\pm$.001	.707* $\pm$.003
	NDCG@5	.020	.053 $\pm$.001	.511* $\pm$.002	.508* $\pm$.002	.508* $\pm$.001	.498* $\pm$.002
	NDCG@10	.026	.067 $\pm$.001	.541* $\pm$.002	.537* $\pm$.002	.537* $\pm$.001	.527* $\pm$.002

5.3 RQ2: Can sequential recommender models utilize non-item pages for predicting the next item?

Our next objective is to investigate whether sequential recommender models can effectively utilize non-item pages. To accomplish this, we use the SynDS datasets based on ML-20m with different variants as introduced in Section 4: Prev-SynDS with non-item pages contains the exact genre information for the next movie and Group-SynDS includes fewer non-item pages with the genre information for the following movies condensed. In addition, we evaluate Random-SynDS non-item-pages with pure noise and Random-(X)-SynDS with varying percentage of noisy and meaningful non-item pages. For all of the artificial pages, we either generate content-based ids by concatenating categories, or assign one placeholder id and encode the categories as attributes. We train all models for both cases. As baseline, we train a model utilizing the sequence with items only. We also include a most popular baseline given the correct genres of the target movie. We train all models for 100 epochs with early stopping, a batch size of 64, and a maximum sequence length of 200.

5.3.1 SynDS Results. Table 2 illustrates the results of our experiments on both Prev-SynDS and Group-SynDS. The genre-specific most popular baseline already performs nearly half as well as most of the models trained on items only, showing that genres are indeed a good indicator for the next movie. The benefit of including meaningful non-item pages for each movie interaction in Prev-SynDS compared to the models trained on pure item sequences is therefore not surprising, as the genre of a movie is given through the preceding non-item page. Over all metrics we can see significant improvements between the Items-Only baseline and Prev-SynDS with non-item pages represented by ids (CPID). Looking at the HR@10, BERT4Rec has the smallest increase with 0.22, followed by NextItNet with an increase of ≈ 0.52 , while the other models even improve by about 0.70 in HR@10. NDCG is also higher over all models compared to the Items-Only baseline, but the improvements are slightly smaller than those observed for the respective HR. For example, NDCG@10 for BERT4Rec has increased by 0.16, and SASRec, which has the highest improvement of the other models, by 0.47. But in general, all models are able to leverage the additional information in the non-item pages and increase their performance over all metrics.

Using page embeddings (PE) of the attributes instead gives very similar results for LightSANs, NARM and SASRec, with less than 0.04 difference in HR@10 compared to the CPID embedding. For GRU4Rec we see a slight improvement with PE. For CASER and CORE the metrics are lower when using PE, but overall they still show a substantial increase in performance compared to the baseline. Notable exceptions are BERT4Rec and NextItNet. Both models can learn from page embeddings but exhibit a high variance in their performance as the model collapses for some runs. Out of the five seeds, three models are collapsing with a performance worse than the baseline. In the case of NextItNet, only one model is able to reach the performance of the baseline, while the others perform similar to the CPID embedding (0.564 on average). The introduction of page embeddings might make these models more unstable, as we do not see a similar behavior using CPIDs.

Group-SynDS contains the same information as Prev-SynDS, but in a more condensed form. When using CPID, the performance is slightly lower, but still very similar to Prev-SynDS for most models, even though the information is less explicitly given. The big exception here again is BERT4Rec, as the HR@10 reaches 0.624, which is far higher than on Prev-SynDS and an increase more similar to the other models. On Group-SynDS, we can observe that BERT4Rec is also able to leverage page embeddings, with even higher performance than with CPIDs and similar performance over all seeds. For NextItNet, the results are slightly lower than on Prev-SynDS and there is again one collapsing model (on the same seed as before). For NARM and CASER the setting in Group-SynDS and page embeddings even yields the best overall performance.

We conclude that all models are capable of utilizing information from non-item pages, as they show significant improvements. The integration of non-item pages as separate ids yields slightly better results than the integration via attributes (PE) for half of the models, but the differences remain relatively small overall. A reason for the better performance could be the relatively small number of non-item pages (≈ 1.300), as it might be easier for the models to learn a fixed number of ids instead of a more complex embedding space. Some models exhibit instabilities with page embeddings, especially BERT4Rec. In contrast to all other models, this is the only model trained with the cloze task [30], which could be one reason for the different behavior. NextItNet with page embeddings also shows a high variance. While the model overall achieves good results, one specific seed gives results similar to the baseline. In general using page embeddings has a comparable performance for most models and is a more flexible method, therefore both representations for non-item pages seem to be valid approaches.

We can further observe that some of the baselines perform surprisingly well. While we did not optimize parameters for any models specifically, baseline models like GRU4Rec, CASER as a simpler convolution neural network or SASRec as a transformer-based architecture still perform comparably. SASRec gives the best overall results, which is in line with previous findings of Fischer et al. [8] for training the model with a cross-entropy loss.

5.3.2 Noise in Non-Item Pages. While our previous experiment indicates that models can profit from additional information in the form of non-item pages, in a realistic setting more data will introduce additional noise. We therefore experiment with the Random-(X)-SynDS datasets, which contain different levels of noise. In Figure 5 we show the performance of all models on the datasets ranging from 0% to 100% shuffled non-item pages and the baseline model using items only for one random seed (212).

While performance naturally drops with increased noise in all models, there are clear differences in the amount of noise different models can handle and the steepness and curve of the drop.

An obvious outlier is BERT4Rec, as we can see again the model’s instability. For page embeddings, only models with 10% and 30% shuffled pages are improving in performance. For the id based approach, the first four models of BERT4Rec perform relatively well. In contrast to this, NextItNet appears more stable with only one exception. The models trained with highly informative non-item pages can leverage this information with both page embeddings and id representation. The performance drops in a steep curve until 50% shuffled pages are reached, but even with more noise the models surpass the baseline. The drop with CASER is also of similar steepness, but overall the model achieves a slightly higher performance. NARM and GRU4Rec show a more linear decline with a sharper drop in performance at about 70%. LightSANs and SASRec even show improved performance up to 80% of shuffled non-item pages. CORE is in between, as CPID and PE embeddings perform quite differently.

We can also observe differences between models with page embeddings and CPIDs. For CASER and CORE the integration with page ids performs better. We can explain this behavior for CORE, as the model explicitly links encoding and decoding for the items, which our page embedding will most likely interfere with. For LightSANs and SASRec CPIDs perform better with more noise. The other results are mixed, but for GRU4Rec page embeddings are performing slightly better.

Overall, we can see that all models are able to handle a reasonable amount of noisy non-item pages. For most models the added noise even has no negative impact, as the performance is still similar to the items-only baseline. In a more realistic setting, non-item pages would be selected avoiding such high levels of noise. Therefore we would not expect a decrease in performance from non-item pages. The complete results are reported in Table 8 for CPID and Table 7 for page embeddings.

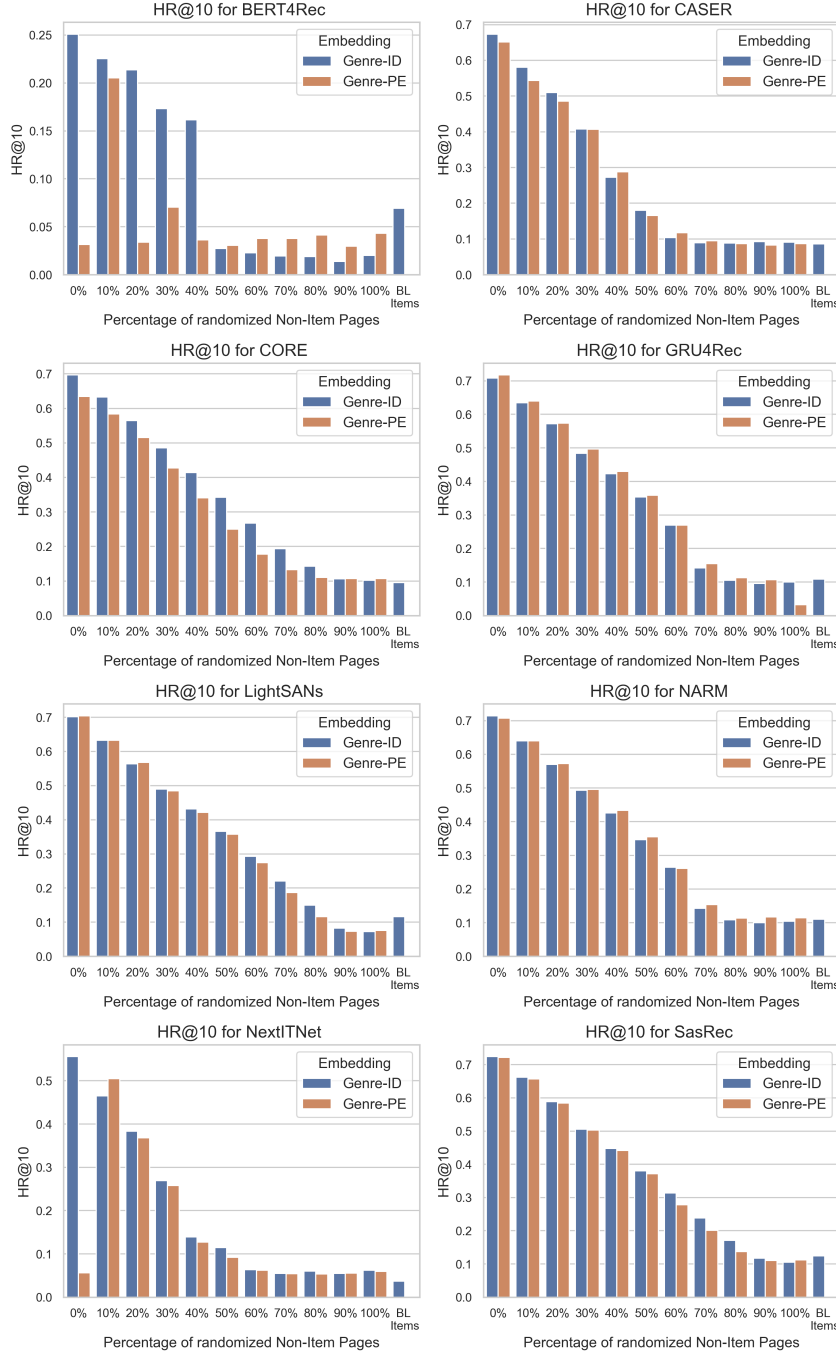


Fig. 5. HR@10 for models on Rand-SynDS with seed 212 on datasets with a growing percentage of shuffled non-item pages. Genre-ID shows the performance of models including non-item pages based on genres with an ID embedding, while Genre-PE shows those with an Page Embedding. The baseline utilizing items only is marked by “BL Items”.

5.3.3 Visualizing Items and Non-Items in the Embedding Space. Using non-item pages in our models can have a high impact on performance. To gain some understanding of the changes happening, we take a look at the item embeddings, as the item embedding layer is where we include non-item pages. We visualize the item embeddings with t-SNE [21] exemplarily in Figure 6 for one seed. To make patterns more visible, we exemplarily highlight movies containing the genres “Comedy” and “Drama”. We also add the average of all “Comedy|Drama” movies and highlight the non-item page representations. In case of page embeddings we display the generic non-item page embedding and add the respective genre embeddings.

While the visualization of the baseline version of SASRec shows only few distinctive structures, e.g. the *hook*-like structure on the top left corner, the item embeddings of SASRec with both CPID and page embeddings reveal clearer patterns. Both show two bigger clusters containing “Comedy|Drama” movies, and we can find more similar structures for both genres. They also contain the hook-like structure as in the baseline models. Overall, the item embeddings seem to have some similarities to each other, and the non-item pages seem to help distinguishing items of different genres better. Interestingly, the structures seem to be consistent even when looking at different models, like GRU4Rec in Figure 6d. Here, we can also find some of the structures mentioned above, like the hook and the bigger, genre specific clusters. We can find similar pattern in most visualizations of the other models and variants, too. We include these figures in our git repository for a complete overview.

Some noticeable exceptions of these patterns can be found when visualizing models which severely underperformed. We visualize both BERT4Rec and NextItNet with page embeddings in Figure 7, which both perform similar to their items-only baselines despite added non-item pages. Both figures show no genre specific patterns at all. Some distinct structures are visible, but the difference to other models with non-item pages is evident. Overall, for models successfully leveraging non-item pages the t-SNE visualisations display distinct clusters of movies sharing genres and genre combinations, while the figures for baseline models and ‘non-successful’ runs do not capture the same patterns.

5.3.4 Item Embedding Similarity. As interpreting t-SNE visualizations can be subjective [33], we want to verify the similarities and dissimilarities for different models in the item embedding space in a more tangible way. While we cannot directly compare item embeddings, we can measure the differences in item similarities between two models. We assume similar item embeddings should keep the semantics between items, for example an item pair closely related in one embedding space should also be closely related in the other embedding space. To measure the difference in item similarities between different models, we first compute the cosine similarity between all items for each model. The result is a matrix of item-to-item similarities, which we normalize. To compare two models, we calculate the absolute difference for all item-to-item similarities between the models and report the average as a measure of similarity between the two embeddings.

In Figure 8 we show the difference in similarity for SASRec and BERT4Rec with and without non-item pages. We show the similarity between the base model and models with CPID and page embeddings as well as a randomly generated embedding. We can confirm our earlier observation: For SASRec all item embedding variants produce partially similar item embeddings compared to the randomly generated embedding. Therefore, some item properties are shared between all models, but we can also see that the models including non-item pages are a bit closer to each other. For BERT4Rec we can also see how the embeddings are in general similar, but that the item embeddings with PE are actually closer to those of the items-only baseline, and dissimilar to the CPID embeddings, which matches the performance of the models for this seed. Similar patterns can be found for the other models, as depicted in Figure 15.

We can also compare the similarity of item embeddings for different models, as shown in Figure 9b.

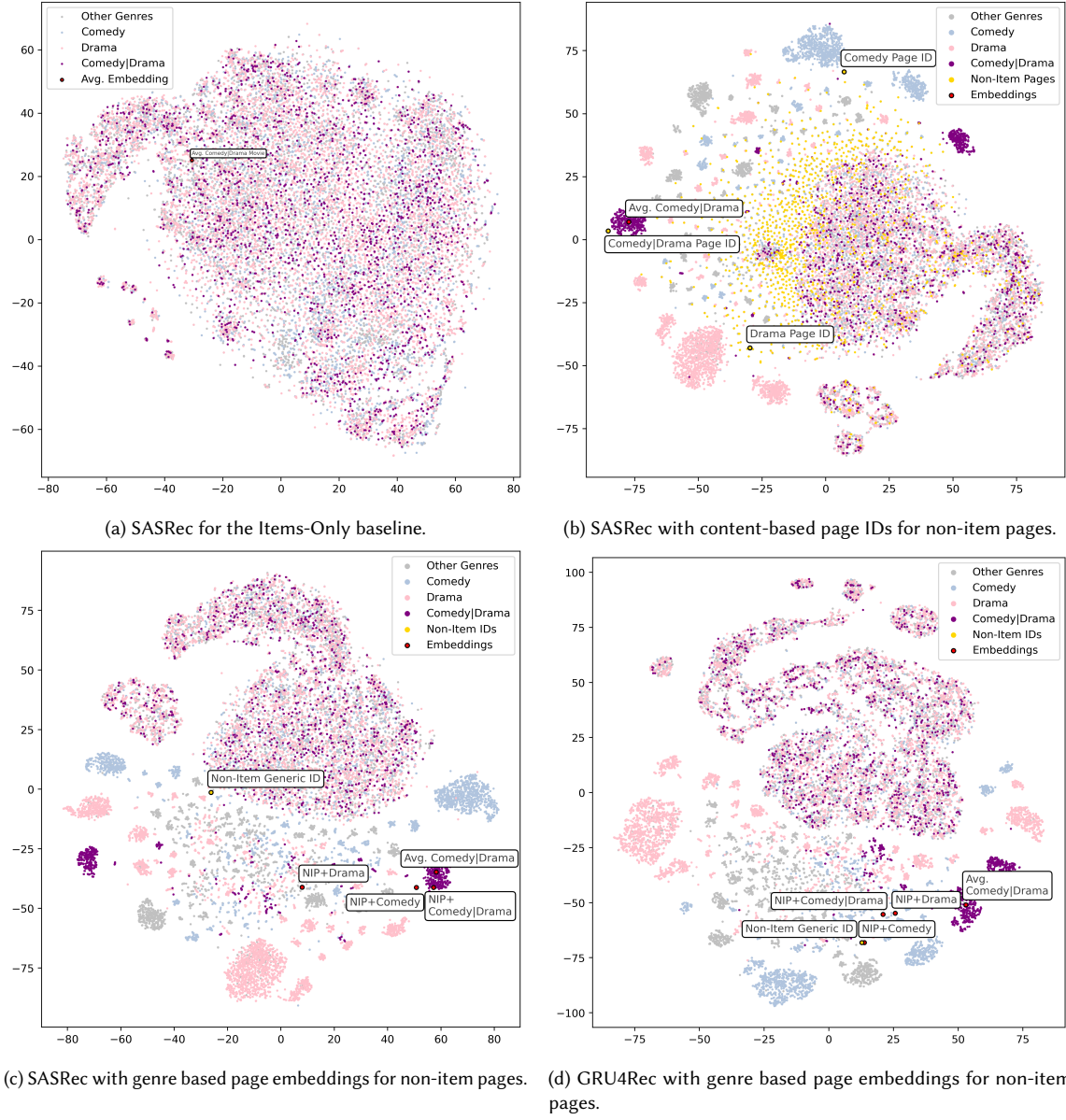


Fig. 6. t-SNE visualization of item embedding space of different model variants on SynDS and Prev-SynDS with highlighted movies containing the genres “Comedy”, “Drama”.

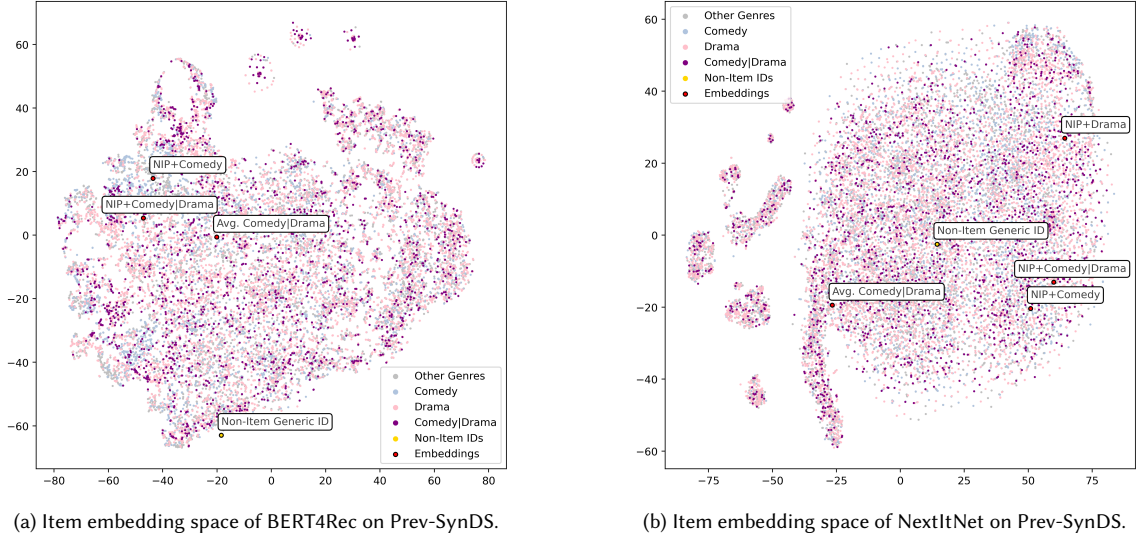


Fig. 7. t-SNE visualization of item embedding space of ‘collapsed’ runs of BERT4Rec and NextItNet on Prev-SynDS with page embeddings.

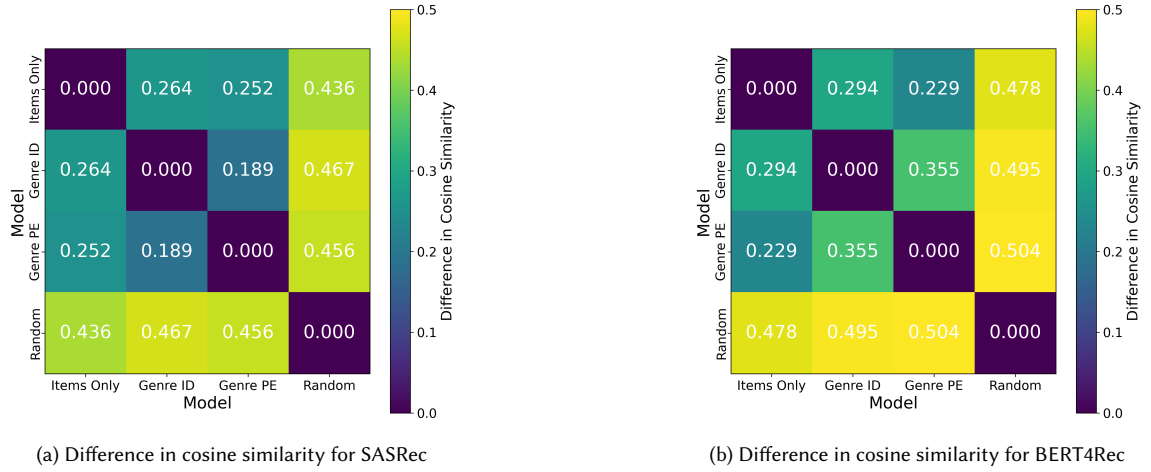


Fig. 8. Difference in Item-to-Item cosine similarity between different embedding variants for seed 212 on (Prev-)SynDS. We report the average of the absolute difference in Item-to-Item similarity between models and a random embedding. A lower score means item-item relations are more similar.

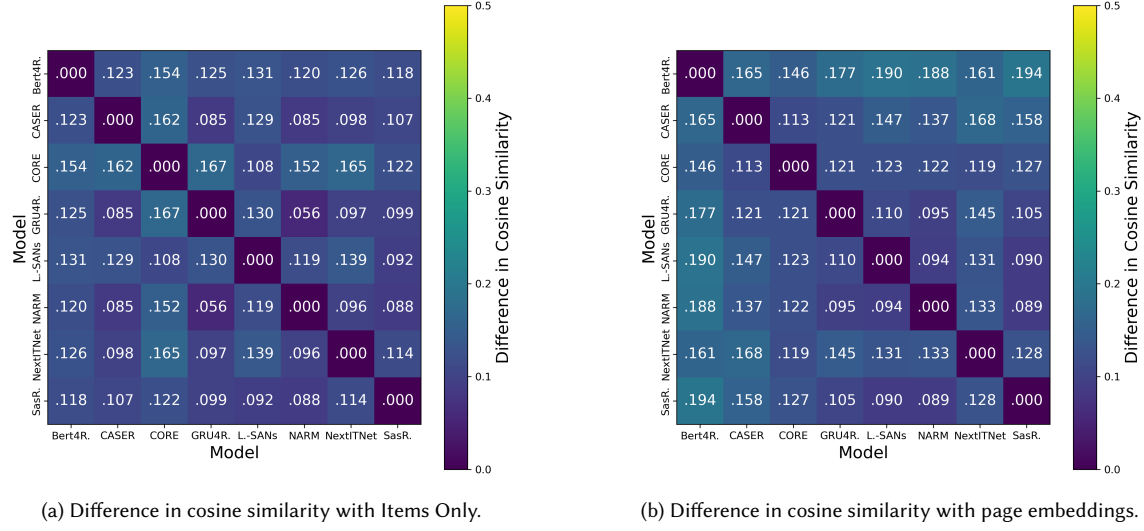


Fig. 9. Difference in Item-to-Item cosine similarity between different embedding variants on (Prev-)SynDS. We report the average of the absolute difference in Item-to-Item similarity between models and a random embedding. A lower score means item-item relations are more similar.

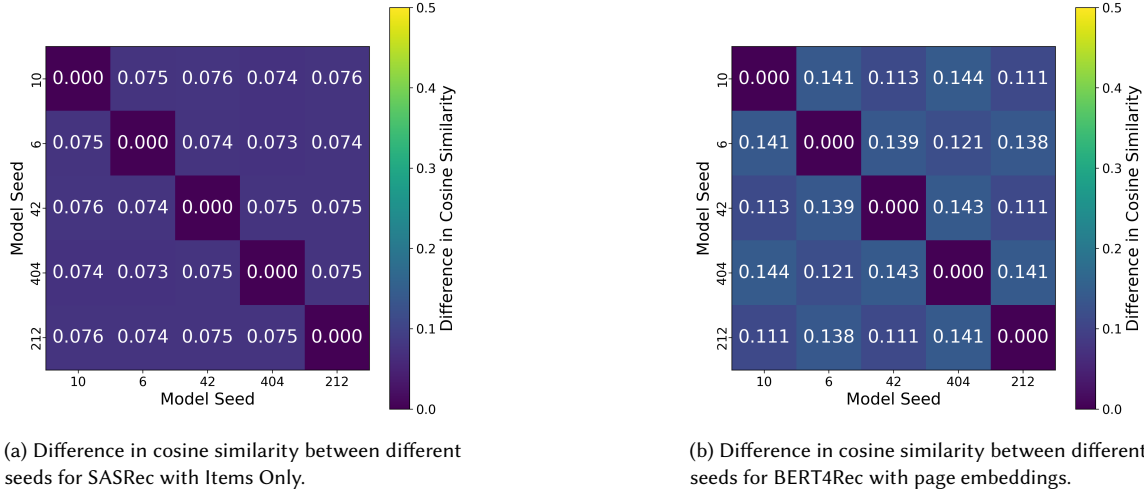


Fig. 10. Difference in Item-to-Item cosine similarity between different embedding variants on (Prev-)SynDS. We report the average of the absolute difference in Item-to-Item similarity between models and a random embedding. A lower score means item-item relations are more similar.

Table 3. Average Hitrate and NDCG with standard deviation over five random seeds for non-item models with Unique Page IDs (UPID) and items-only baseline models (Items-BL) on the Coveo-Pageview dataset. Results marked with * show a significantly higher performance compared to the baseline for $p < 0.01$.

Models	Metrics	Items-BL	UPID	Models	Metrics	Items-BL	UPID
BERT4Rec	HR@1	.082 ± .002	.080 ± .004	LightSANs	HR@1	.127 ± .003	.126 ± .002
	HR@5	.286 ± .003	.284 ± .003		HR@5	.311 ± .002	.328* ± .002
	HR@10	.360 ± .003	.361* ± .003		HR@10	.386 ± .002	.408* ± .001
	NDCG@5	.187 ± .003	.185 ± .003		NDCG@5	.222 ± .001	.230* ± .002
	NDCG@10	.211 ± .003	.210 ± .003		NDCG@10	.247 ± .001	.256* ± .001
Caser	HR@1	.093 ± .001	.083 ± .004	NARM	HR@1	.097 ± .054	.044 ± .060
	HR@5	.288 ± .002	.272 ± .004		HR@5	.251 ± .140	.124 ± .169
	HR@10	.360 ± .002	.347 ± .003		HR@10	.311 ± .173	.157 ± .213
	NDCG@5	.195 ± .002	.181 ± .004		NDCG@5	.177 ± .098	.085 ± .116
	NDCG@10	.218 ± .002	.206 ± .003		NDCG@10	.196 ± .109	.096 ± .130
Core	HR@1	.142 ± .005	.113 ± .007	NextIttNet	HR@1	.138 ± .002	.111 ± .003
	HR@5	.403 ± .001	.381 ± .001		HR@5	.317 ± .003	.301 ± .002
	HR@10	.476 ± .001	.460 ± .002		HR@10	.391 ± .003	.381 ± .002
	NDCG@5	.281 ± .002	.256 ± .004		NDCG@5	.230 ± .002	.209 ± .002
	NDCG@10	.305 ± .003	.282 ± .004		NDCG@10	.254 ± .002	.235 ± .003
GRU4Rec	HR@1	.066 ± .003	.091* ± .006	SASRec	HR@1	.140 ± .006	.127 ± .002
	HR@5	.228 ± .002	.268* ± .003		HR@5	.334 ± .002	.333 ± .002
	HR@10	.298 ± .002	.343* ± .002		HR@10	.410 ± .001	.414* ± .002
	NDCG@5	.150 ± .001	.182* ± .004		NDCG@5	.240 ± .004	.233 ± .001
	NDCG@10	.172 ± .001	.206* ± .004		NDCG@10	.265 ± .003	.260 ± .001

5.4 RQ3: Are there gains from including non-item pages without content representations?

As we have seen that embedding pages via ids is beneficial in an artificial setting, we aim to verify this with the Coveo-Pageview dataset on real-life data. In this dataset, the only available representation for non-product interactions is a unique hashed URL, leaving no alternative method for creating a different representation. Using the URL as a unique page id gives us the results shown in Table 3. We train all models for 50 epochs with a batch size of 64 and a maximum sequence length of 30.

5.4.1 Results. The influence of non-item pages on the performance varies between models. For CASER, CORE and NextIttNet the performance drops with non-item pages. The same seems to be the case for NARM, but the model is also very unstable on this dataset: three models with non-item pages and one baseline model collapse and report a HR@10 $\leq .002$. Excluding these runs gives an average HR@10 of .389 for the baseline and UPID at a very similar performance with .391. Small differences are also found in BERT4Rec and SASRec, with small decreases but also increases in some metrics. Only for LightSANs and GRU4Rec we see a clear improvement in all metrics, e.g., of .022 and .045 in HR@10. One reason for this might be the number of non-item pages, which is more than twice as many as the number of items in the Coveo-Pageview dataset (Table 1) and therefore their usage is very sparse.

The growing vocabulary might also hinder the models, as already discussed in Section 3.3 in more detail. As we do not have further information on the types of non-item pages in the dataset, it is also possible that the non-item pages contain too little useful information and too many random and noisy pages. Interestingly, all models that exhibited a greater performance decline (CASER, CORE, NextIttNet) are models with a steeper drop in performance in our experiment

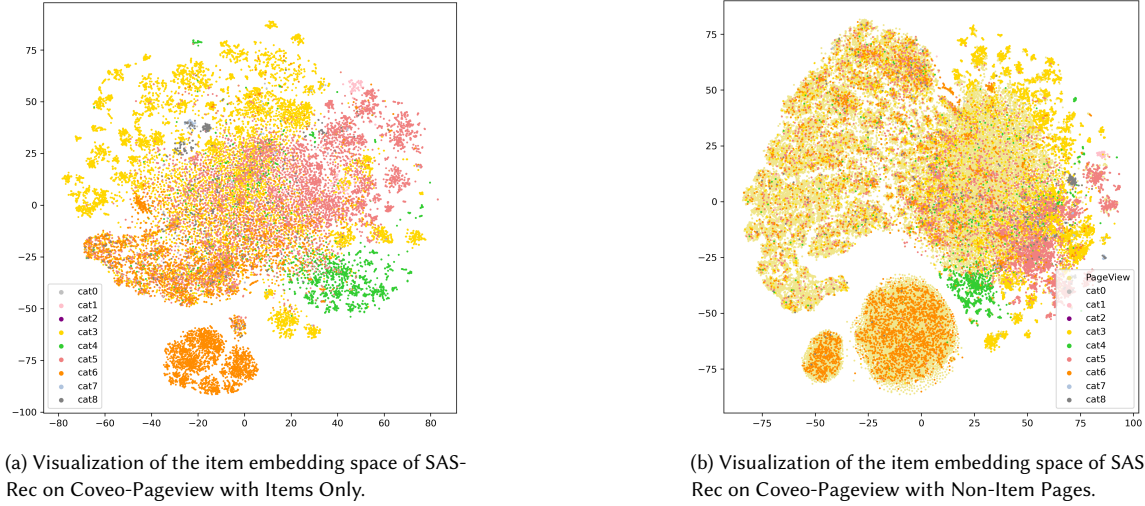


Fig. 11. t-SNE visualization of item embedding space of SASRec on Coveo-Pageview with CPID embeddings.

with added noise (see Section 5.3.2), supporting our finding that they are less effective at handling noisy interactions compared to the others. Overall, there is only a small benefit of using the non-item pages here, as only two models can extract useful behavior patterns, but the majority does not improve.

5.4.2 Visualization and Item Embedding Similarity. We visualize exemplary the item embedding space for SASRec with and without Non-Item Pages in Figure 11 on seed 212. We can see some recurring patterns in both, based on the top-level categories of the items. We also see the huge amount of non-item pages in this dataset. They also form patterns, for example around the bigger cluster of “cat6” on the bottom, which means that they are probably related to these items. In most model visualizations we can identify some similar clusters, but the item embedding space looks quite different for models, which have a performance close to zero, for example a run of NARM without pageviews (see Figure 17).

We also look at the difference in cosine similarity for different models and item embeddings. We observe a similar overall pattern as before in the SynDS dataset: The difference in cosine similarity between our embedding variants is bigger than the difference between different models. We exemplary plot these for SASRec and models with page embeddings in Figure 12. Between the models with page embeddings, CORE and GRU4Rec show bigger differences to most other models. CORE is by far the best performing model, while GRU4Rec has a huge improvement through non-item pages, which both seems to create more different item embeddings. The CORE model also has a high dissimilarity in the items only setting and is also the best performing model there, which suggests that it is able to capture aspects of the data the other models are missing. Additional visualizations are included in our repository.

5.5 RQ4: Can we find good representations for utilizing search requests as non-item interactions?

In this experiment, we investigate content-based page representations. In the Coveo-Search dataset, non-item interactions are introduced by search requests. A search request represents a clear expression of the user’s intent and should contain

Table 4. Average Hitrate and NDCG with standard deviation over five random seeds for non-item models with Unique Page IDs (UPID) and items-only baseline models (Items-BL) on the Coveo-Search dataset. We represent search events by the most frequent categories (FreqC-) or the categories of the first result (FirstC-) as page embeddings (PE) and id embeddings (ID). We also use the id embedding of the first item in the result (1.Item-ID) and the page embedding of the query (Query-PE). Results marked with * show a significantly higher performance compared to the baseline for $p < 0.01$ and results marked with + for $p < 0.05$.

Model	Metrics	Items-BL	FreqC-ID	FreqC-PE	FirstC-ID	FirstC-PE	1.Item-ID	Query-PE
BERT4R.	HR@1	.071±.001	.073⁺ ±.002	.068±.003	.073 ⁺ ±.001	.072±.003	.070±.002	.073 ⁺ ±.002
	HR@5	.246±.002	.261 [*] ±.002	.252 [*] ±.005	.261 [*] ±.002	.258 [*] ±.002	.225±.003	.263[*] ±.004
	HR@10	.318±.003	.334⁺ ±.004	.323 [*] ±.005	.334 [*] ±.004	.329 [*] ±.004	.289±.003	.332 [*] ±.005
	NDCG@5	.161±.001	.169 [*] ±.002	.162 ⁺ ±.003	.169 [*] ±.002	.168 [*] ±.003	.150±.001	.170[*] ±.003
	NDCG@10	.184±.001	.193 [*] ±.003	.185 ⁺ ±.004	.193 [*] ±.003	.191 [*] ±.003	.170±.001	.193[*] ±.003
Caser	HR@1	.078±.002	.071±.002	.071±.003	.072±.003	.070±.001	.071±.002	.071±.003
	HR@5	.238±.002	.224±.004	.221±.001	.226±.003	.222±.003	.206±.002	.224±.002
	HR@10	.302±.003	.290±.003	.286±.002	.291±.002	.288±.002	.260±.002	.290±.003
	NDCG@5	.161±.002	.151±.003	.148±.001	.152±.003	.149±.001	.141±.001	.150±.003
	NDCG@10	.182±.002	.172±.002	.169±.002	.173±.002	.170±.001	.158±.001	.172±.003
Core	HR@1	.112±.004	.093±.004	.100±.002	.093±.002	.101±.002	.092±.003	.103±.004
	HR@5	.353±.001	.343±.001	.341±.002	.343±.001	.343±.002	.309±.002	.345±.003
	HR@10	.421±.001	.414±.001	.411±.002	.415±.002	.414±.001	.370±.001	.421±.002
	NDCG@5	.240±.002	.227±.002	.228±.001	.227±.001	.230±.002	.208±.001	.232±.002
	NDCG@10	.262±.002	.250±.002	.251±.001	.250±.001	.253±.001	.228±.001	.257±.002
GRU4R.	HR@1	.052±.001	.057 [*] ±.004	.074 [*] ±.003	.056 [*] ±.004	.079[*] ±.003	.056 [*] ±.005	.075 [*] ±.006
	HR@5	.185±.007	.193 [*] ±.007	.212 [*] ±.009	.193 [*] ±.006	.219 [*] ±.007	.178±.005	.223[*] ±.005
	HR@10	.249±.009	.258 [*] ±.006	.276 [*] ±.008	.259 [*] ±.006	.286 [*] ±.005	.237±.005	.291[*] ±.005
	NDCG@5	.120±.004	.126 [*] ±.005	.144 [*] ±.006	.126 [*] ±.005	.150 [*] ±.004	.118±.005	.150[*] ±.005
	NDCG@10	.141±.004	.148 [*] ±.005	.165 [*] ±.006	.148 [*] ±.005	.172 [*] ±.004	.138±.005	.173[*] ±.005
L-SANs	HR@1	.105±.011	.097±.004	.106±.006	.100±.003	.098±.010	.092±.006	.106±.003
	HR@5	.278±.008	.282 [*] ±.003	.286 [*] ±.004	.282 [*] ±.002	.288 [*] ±.004	.254±.004	.295[*] ±.001
	HR@10	.346±.007	.352 [*] ±.002	.358 [*] ±.002	.353 [*] ±.002	.358 [*] ±.002	.315±.004	.370[*] ±.002
	NDCG@5	.194±.009	.192±.002	.198 [*] ±.005	.194±.002	.196 [*] ±.007	.176±.003	.204[*] ±.002
	NDCG@10	.216±.009	.215±.002	.222 [*] ±.004	.217±.002	.219 [*] ±.006	.196±.003	.228[*] ±.002
NARM	HR@1	.072±.003	.077 [*] ±.002	.081 [*] ±.003	.077 [*] ±.001	.079 [*] ±.003	.063±.035	.085[*] ±.003
	HR@5	.224±.002	.239 [*] ±.003	.236 [*] ±.002	.239 [*] ±.003	.236 [*] ±.002	.175±.098	.244[*] ±.004
	HR@10	.292±.002	.308 [*] ±.003	.306 [*] ±.003	.307 [*] ±.003	.304 [*] ±.002	.224±.124	.314[*] ±.002
	NDCG@5	.150±.002	.160 [*] ±.002	.160 [*] ±.002	.160 [*] ±.002	.160 [*] ±.002	.120±.067	.167[*] ±.002
	NDCG@10	.172±.002	.183 [*] ±.002	.183 [*] ±.003	.183 [*] ±.002	.182 [*] ±.002	.136±.076	.190[*] ±.002
NexItNet	HR@1	.071±.003	.069±.002	.055±.005	.069±.001	.042±.031	.067±.002	.067±.002
	HR@5	.209±.003	.210±.003	.172±.013	.210±.004	.128±.089	.194±.004	.207±.002
	HR@10	.273±.002	.274±.004	.233±.013	.274±.004	.172±.111	.249±.005	.273±.003
	NDCG@5	.142±.002	.141±.002	.115±.009	.141±.002	.086±.060	.132±.003	.139±.002
	NDCG@10	.163±.002	.162±.003	.135±.009	.162±.002	.100±.068	.150±.003	.160±.002
SASRec	HR@1	.097±.007	.097±.005	.102 [*] ±.005	.097±.005	.101 [*] ±.009	.090±.004	.103[*] ±.012
	HR@5	.293±.002	.284±.002	.290±.002	.285±.002	.288±.003	.261±.001	.293±.003
	HR@10	.364±.002	.357±.002	.360±.002	.357±.002	.359±.003	.321±.002	.366⁺ ±.003
	NDCG@5	.199±.004	.193±.003	.199±.002	.193±.003	.197±.004	.179±.002	.201[*] ±.006
	NDCG@10	.222±.003	.217±.003	.222±.002	.217±.003	.220±.003	.198±.002	.225[*] ±.006

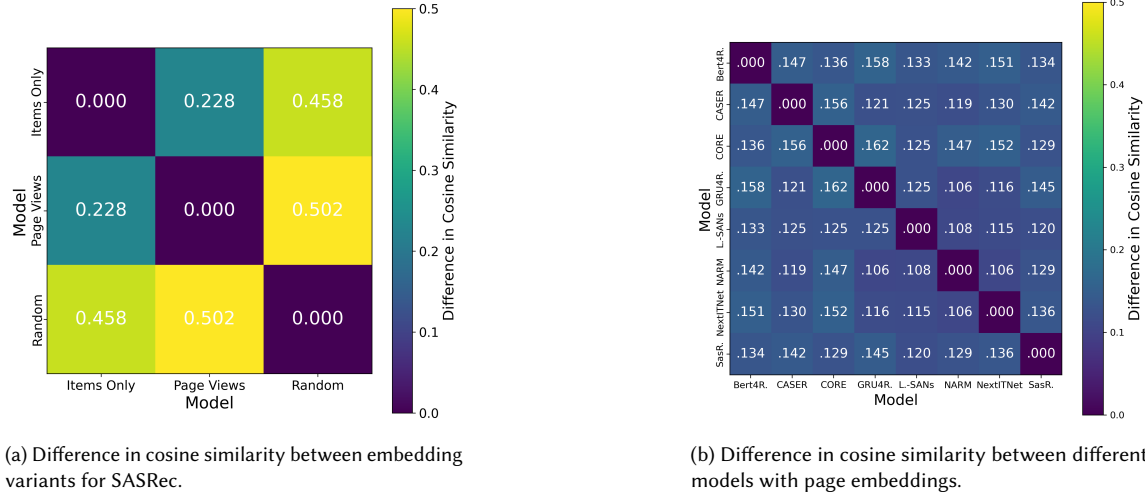


Fig. 12. Difference in Item-to-Item cosine similarity between different variants on Coveo-Pageview. For each model the cosine similarity between all items is computed and normalized. We report the average of the absolute difference in similarity between two models. A lower score means item-item relations are similar in both models. A randomly generated embedding is included as a baseline.

valuable information. We compare three different CPID tokens from the list of retrieved items to represent a search interaction:

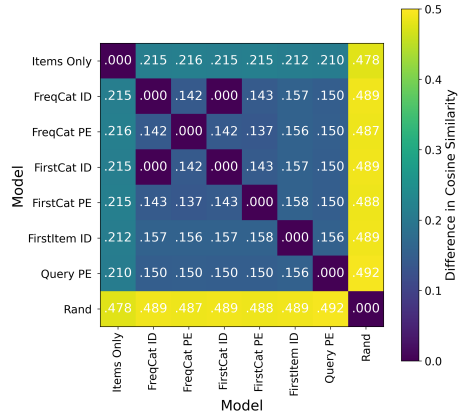
(a) the category, (b) the product id of the first item, and (c) the concatenation of the three most frequent categories in the list. In case of product id, no new ids are added to the vocabulary. For both categories, we also test the attribute-based approach with multi-hot embeddings (PE) based on the categories instead of unique ids. In addition, we also evaluate using the provided query embedding, as it directly represents the search intent and not the feedback from the search engine. We use a batch size of 64, a maximum sequence length of 30 and train for 100 epochs.

Table 4 shows our results. Looking at the best-performing setting of each model, we can see that including the search interactions in BERT4Rec, NARM, LightSANS and GRU4Rec brings significant performance improvements, of at least 1.4 percentage points and up to 4.2 percentage points on the HR@10. We also observe small increases in metrics for SASRec for the embedded query, but they are well within one standard deviation.

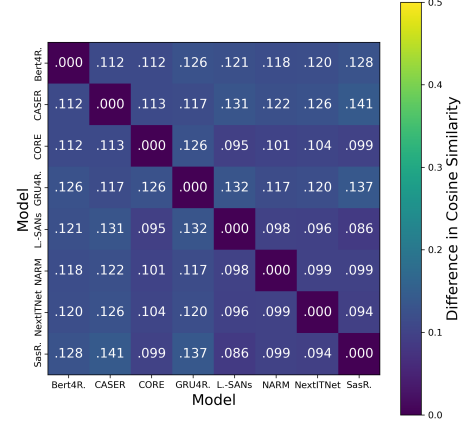
In a similar scope, the baseline performance for NextITNet is better compared to the query or category id embeddings. But there are also two models, CASER and CORE, where the baseline is clearly outperforming all models containing non-item pages, with the performance dropping visibly for all non-item variants.

We can also observe common patterns regarding representations of search interactions. The first item in the results list (FirstItem-ID) hurts the performance of all models and is therefore not a fitting representation for a search request. Between models using the most frequent or first category we observe similar metrics for the same type of embedding.

But between CPID and PE embeddings there are notable differences: NARM and BERT4Rec show slightly higher performance when using CPID, while LightSANS, and GRU4Rec seem to work better with page embeddings. However, the four models show improvements for all variants of category-based representations. By far the biggest improvement is achieved when using a query embedding to represent the search. GRU4Rec has the highest gains in metrics and for NARM, LightSANS and BERT4Rec the query embedding is overall the most beneficial way of including the search as



(a) Difference in cosine similarity between embedding variants for SasRec.



(b) Difference in cosine similarity between different models with the search query embedding.

Fig. 13. Difference in Item-to-Item cosine similarity between different variants on Coveo-Search. For each model the cosine similarity between all movies is computed and normalized. We report the average of the absolute difference in similarity between two models. A lower score means item-item relations are similar in both models. A randomly generated embedding is included as a baseline.

well. This can be explained by the fact that the query embedding is a direct representation of the search request. In contrast, other representations are dependent on retrieved items and, therefore, on the quality of the search engine and its interpretation of the user query. Here, the advantage of using more flexible page embeddings becomes evident, as CPID embeddings cannot represent a search request directly. We can also observe that the three models (CASER, CORE and NextITNet), which do not benefit from the inclusion of the search non-item pages, are all showing lower improvements on highly noisy data in Section 5.3.2, and the signal from the search could be too weak for them.

5.5.1 Visualization and Item Embedding Similarity. The item embedding spaces visualized with t-SNE reveal patterns similar to those of the models in the Coveo-Pageview dataset, as they are both extracted from the Coveo challenge data (see Figure 18 and more in our repository). Overall, item embeddings show similar patterns for all models, which we verify by plotting the difference in cosine similarity. Figure 13a shows the difference in cosine similarity for SASRec. We can observe that the Items-Only model differs the most from all other models. Although not all embedding variants improve performance, each contributes to a distinct change in the item embedding space. We observe this pattern with all model architectures. Meanwhile, the differences between models with any kind of search representation are less pronounced, with the most relevant difference in the item embedding space occurring between including non-items or not including them.

We also confirm the same pattern as observed earlier: Adding non-items has a greater impact on the item embedding space than changing the model architecture, as shown in Figure 13a for query embeddings.

5.6 RQ5: Can we find good representations to integrate list pages as non-item pages?

We further explore the potential benefits of integrating non-item pages with content-based page ids and page embeddings for list pages. In the Fashion dataset, categories and filters set by users are available as direct descriptions of a list page’s

Table 5. Hitrate and NDCG for non-item models and items-only baseline models (Items-BL) on the Fashion dataset for seed 212. We represent list pages based on the categories (Cat-) or the filters (Filter-) as page embeddings (PE) and content-based page id embeddings (CPID). We also use the id embedding of the first item on a list page directly (First-ID). Results marked with * show a significantly higher performance compared to the baseline for $p < 0.01$ and results marked with + for $p < 0.05$.

Model	Metrics	Item Only	First-ID	Cat-CPID	Cat-PE	Filter-CPID	Filter-PE
BERT4Rec	HR@1	.040	.092*	.082*	.041	.082*	.030
	HR@5	.279	.308*	.295*	.195	.301*	.170
	HR@10	.380	.397*	.385*	.289	.393*	.254
	NDCG@5	.161	.203*	.191*	.119	.194*	.101
	NDCG@10	.194	.232*	.220*	.149	.224*	.128
Caser	HR@1	.020	.082*	.076*	.087*	.080*	.089*
	HR@5	.128	.260*	.247*	.274*	.254*	.282*
	HR@10	.196	.341*	.329*	.358*	.336*	.368*
	NDCG@5	.075	.173*	.164*	.183*	.169*	.188*
	NDCG@10	.097	.200*	.190*	.210*	.196*	.216*
Core	HR@1	.104	.101	.113*	.139*	.107+	.107+
	HR@5	.352	.415*	.417*	.420*	.423*	.427*
	HR@10	.438	.503*	.504*	.507*	.511*	.513*
	NDCG@5	.235	.269*	.275*	.288*	.276*	.278*
	NDCG@10	.263	.297*	.303*	.316*	.304*	.306*
GRU4Rec	HR@1	.031	.085*	.088*	.102*	.092*	.094*
	HR@5	.183	.270*	.282*	.306*	.291*	.298*
	HR@10	.265	.354*	.365*	.388*	.375*	.385*
	NDCG@5	.108	.181*	.188*	.207*	.195*	.200*
	NDCG@10	.134	.208*	.215*	.234*	.222*	.228*
LightSANS	HR@1	.084	.153*	.165*	.142*	.170*	.155*
	HR@5	.331	.395*	.401*	.386*	.405*	.399*
	HR@10	.430	.478*	.483*	.469*	.490*	.484*
	NDCG@5	.210	.280*	.288*	.269*	.293*	.282*
	NDCG@10	.242	.306*	.315*	.297*	.321*	.309*
NARM	HR@1	.038	.084*	.001	.121*	.092*	.089*
	HR@5	.181	.274*	.001	.326*	.289*	.293*
	HR@10	.264	.358*	.003	.408*	.375*	.378*
	NDCG@5	.111	.182*	.001	.227*	.194*	.194*
	NDCG@10	.137	.209*	.002	.253*	.222*	.222*
NextItNet	HR@1	.010	.056*	.042*	.052*	.025*	.069*
	HR@5	.083	.189*	.145*	.174*	.093*	.221*
	HR@10	.134	.269*	.213*	.251*	.147*	.308*
	NDCG@5	.047	.124*	.094*	.114*	.060*	.146*
	NDCG@10	.063	.150*	.116*	.139*	.077*	.175*
SASRec	HR@1	.076	.154*	.159*	.123*	.159*	.151*
	HR@5	.327	.385*	.394*	.386*	.399*	.403*
	HR@10	.430	.472*	.477*	.473*	.484*	.491*
	NDCG@5	.205	.275*	.282*	.260*	.284*	.282*
	NDCG@10	.238	.303*	.309*	.288*	.312*	.311*

content. We test both CPID and PE-based approaches for categories and filters. The first item on a list page is also used as a representation. We use a batch size of 64, a maximum sequence length of 50 and train for 10 epochs.

We present our results in Table 5. We see significant improvements for all models and metrics when adding non-item pages. For example the HR@10 for NextItNet more than doubles and even models that were already performing well show improvements, e.g., CORE increasing on HR@10 from 43.8% up to 51.3%. Adding list pages achieves higher metrics across all experiments, regardless of the representation, with three notable exceptions: Using PE embeddings for BERT4Rec and Cat-CPID for NARM reduces the performance below the respective baseline. As the performance drop is quite high, we attribute this to the models instabilities, which we have already discussed before. For BERT4Rec the highest improvement is achieved by representing the list pages by the first item on each page. For GRU4Rec and NARM the best representation is a page embedding of the categories, for CORE the results are inconclusive between category and filters. All other models improve the most when using representations based on the filters of the list page. Overall, most models show higher improvements with page embeddings instead of CPID, though the difference is marginal. There are also higher gains from using filter-based representations for most models. In the case of the Fashion dataset, both category and filter-based representations for list pages perform well. Filters contain more detailed information on the selected categories and therefore seem to be slightly more beneficial. Interestingly, the representation as attributes yields better results than the encoding as an id for this dataset. This might be related to the high number of attributes and their combinations compared to the other datasets.

5.6.1 Item Embedding Similarity. Visualizing the embedding spaces with t-SNE shows similar patterns over all models. We again examine the difference in cosine similarity for the models on the Fashion dataset. We observe the same behavior as before: The inclusion of list pages leads to a higher difference in cosine similarity compared to the different embedding variants for nearly all model architectures (Figure 14a). One exception appears for BERT4Rec, where most variants show a relatively high distance to each other, presumably due to unstable behavior with page embeddings. As noted before, the impact of the model architectures stays relatively low for the different embeddings. We include examples for SASRec and the page embedding with filters models in Figure 14b.

6 Discussion

In this section we consolidate and discuss our overall findings. We discuss performance differences in the models and proposed modeling approaches and compare our results to previous work.

6.1 Model Differences

In the course of our study we find some model architectures able to leverage information from non-item pages better than others. On the SynDS dataset all models increase significantly in their performance, as the signal added through non-item pages is highly indicative for the respective item. For CASER and NextItNet in contrast, we observe a performance drop on both Coveo datasets and for CORE there is a similar trend. For GRU4Rec, LightSANs and NARM we can report an overall positive outcome, aside from the instabilities of NARM on the Coveo-Pageview dataset. For both BERT4Rec and SASRec we face partially inconclusive results with a slight positive trend. On the Fashion dataset, all architectures can improve their performance significantly, likely due to less abstract non-item interactions. Interestingly, this aligns with our findings in Section 5.3.2, where lower performance on the randomized data corresponds to lower improvements on real-life data and a higher resistance to noisy data leads to larger performance gains. CASER and NextItNet are also the only two CNN-based models in our selection. There might be a common property of CNNs

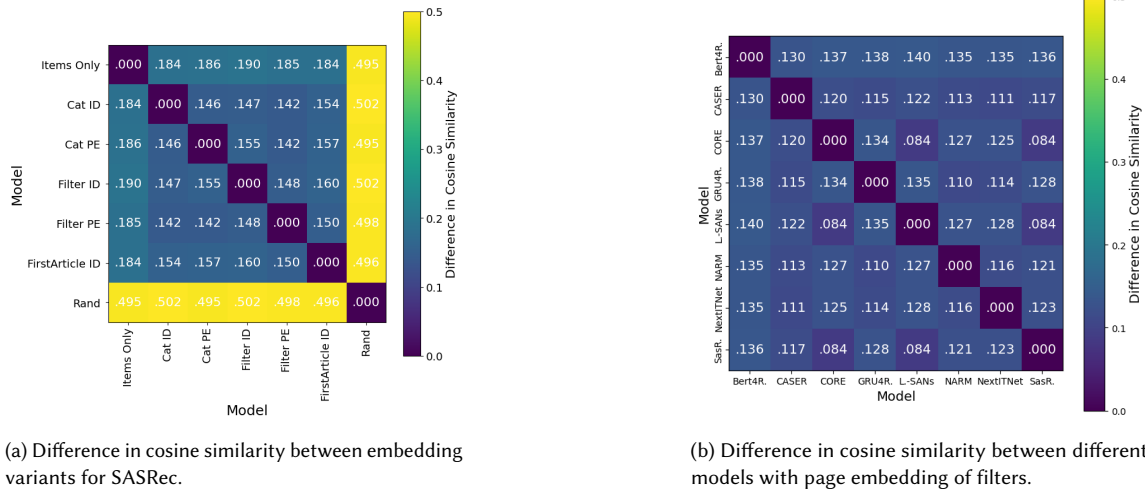


Fig. 14. Difference in Item-to-Item cosine similarity between different variants on Fashion. For each model the cosine similarity between all items is computed and normalized. We report the average of the absolute difference in similarity between two models. A lower score means item-item relations are similar in both models. A randomly generated embedding is included as a baseline.

preventing a more effective inclusion of non-item pages. CORE explicitly couples the embedding space for encoding and decoding, which might interfere with our additions to the item embedding space, especially with page embeddings.

6.2 Sequence Length

Using non-item pages also means that our sequences will grow longer, as observed in the higher average session length in Table 1. Adding non-item pages to a session can sometimes lead to the session exceeding the maximum sequence length, which means that the oldest interactions are dropped, while the corresponding model without non-item pages would include them. Adjusting the maximum sequence lengths might lead to higher recommendation performance, as more interactions are considered, but will also increase the computational cost of the model. For self-attention-based models the influence of the sequence length n on runtime is $O(n^2)$, for recurrent and convolutional architectures $O(n)$ [32]. For example, by adding a non-item interaction for each item interaction, the cost would increase to 4 and 2 times of the original, respectively. Depending on the proportions of non-item interactions in the dataset, the impact on efficiency can be high and should be considered while optimizing for performance. In our experimental setup we did not change or optimize the maximum sequence length for our models. Although this approach results in the loss of some interactions, the advantage of incorporating more recent non-item pages appears to outweigh the impact of missing older interactions.

6.3 Non-Item Representations

Between the different representations for non-items, we can also observe patterns depending on the models, but also on the datasets. For CORE CPIDs work better in all experiments, because of the model's structure. The experiments with randomized data show an overall preference for adding IDs, except for GRU4Rec and NARM. On Coveo-Search in contrast, we see a clear preference for the query page embedding. Between the category-based embeddings the results

are mixed, similar to the results on the Fashion dataset. Overall, the choice between CPIDs and page embeddings seems to depend on how informative and how complex the non-item page representation is.

Models utilizing CPIDs work better as long as the number of attributes and therefore added CPIDs stays relatively low, as in SynDS or the category-based representations in Coveo-Search. But once a huge number of Non-Item Pages is added as CPIDs and the vocabulary increases (as in Fashion), page embeddings become more efficient. Even if it is more difficult for models to learn from embeddings than explicit IDs, there seems to be a turning-point where adding more IDs becomes a disadvantage.

There is also the factor of the actual informativeness of the content for a representation: The query embedding in Coveo-Search contains far more accurate information about users’ intents than some average categories and is therefore more helpful. In general, noticeable improvements occur only when the information gain from non-item pages is substantial and clear. For example, the information content of non-item pages in the Coveo-Pageview dataset seems to be relatively low and only the models most responsive to non-item pages can leverage it. Overall, selecting useful non-item pages and appropriate means of representation is an important step when including non-item pages. For non-item pages represented by CPIDs, HypTrails [25] can be employed in advance to examine the influence on following items.

6.4 Comparison with Previous Results

Table 6. Hitrate and NDCG for BERT4Rec and SASRec models on the Prev-SynDS and Group-SynDS datasets for page embeddings (-PE) and id embeddings (-ID), as well as the items-only baseline (Items BL). “RecB” marks the results from this paper based on RecBole, averaged over 5 different seeds, while “ASME” shows the respective results reported by [7] on one seed.

Model	Metrics	Items BL		Prev-ID		Prev-PE		Group-ID		Group-PE	
		RecB	ASME	RecB	ASME	RecB	ASME	RecB	ASME	RecB	ASME
BERT4Rec	HR@1	.010	.019	.119	.393	.040	.308	.294	.397	.308	.016
	HR@5	.042	.078	.241	.662	.087	.605	.534	.652	.559	.040
	HR@10	.075	.133	.298	.753	.111	.719	.624	.745	.653	.061
	NDCG@5	.026	.049	.183	.537	.064	.465	.421	.533	.441	.028
	NDCG@10	.037	.066	.201	.566	.072	.503	.450	.563	.471	.035
SASRec	HR@1	.025	.027	.370	.426	.368	.384	.371	.426	.361	.371
	HR@5	.081	.103	.634	.699	.630	.681	.629	.681	.618	.659
	HR@10	.123	.164	.725	.785	.721	.780	.717	.768	.707	.756
	NDCG@5	.053	.066	.511	.572	.508	.543	.508	.563	.498	.525
	NDCG@10	.067	.085	.541	.600	.537	.575	.537	.591	.527	.556

As this paper is a direct extension of the work done in [7], we want to consolidate previous and current findings. In our previous work, we utilized the recommender framework ASME by Dallmann et al. [3] to conduct experiments on SASRec and BERT4Rec. For the current paper, we used RecBole [41] as basis for our experiments, as more sequential recommender models are readily available and the framework is more commonly used. Our experimental settings are the same as in [7].

We exemplarily compare the results on the SynDS dataset for SASRec and BERT4Rec, the common models in both settings and display the results in detail in Table 6.

While we can see the benefits of including non-item pages in both previous and current results, the performance of both models differs between the implementations. For BERT4Rec we only reach half of the performance for the baseline

model. For SASRec the difference is less prominent but still very noticeable. The difference in models with non-item pages is even higher. One reason for this could be the instability of BERT4Rec, especially when comparing models using page embeddings. The fact that our new results are based on multiple seeds can also explain the differences to some extent only, as the standard deviation is not that high (see Table 2).

On the other datasets, the performance for BERT4Rec and SASRec is also lower in RecBole. The overall sentiment, however, is supported by both experiments: On Coveo-Pageview, the benefit of non-item pages is ambiguous. However, on Coveo-Search and Fashion, both models show improvements, with query-based and filter-based representations yielding the greatest gains.

Reproducibility over different implementations has been a known problem especially for BERT4Rec, as recently reported in [23]. The study mentions both, ASME as well as RecBole and reports different performance for both. The implementation of BERT4Rec in RecBole has been updated since, necessitating further investigations: When comparing current implementations in both frameworks, we still found some inconsistencies, for example in the masking task. ASME masks items either with the mask token, a random value or keeps the original value, while RecBole only uses the mask token. The generation of target items is also slightly different, as we noticed that the loss can become NaN for some edge cases in RecBole. We can also point to differences which can explain the performance gaps in SASRec: In ASME the loss is computed for each step of the sequences, not only for the last target item, therefore training on all subsequences in one step. It also uses a separate layer for projecting back to the item space, while the implementation of RecBole reuses the item embedding layer. Overall, independent of the performance of the specific implementations, the results from previous and current experiments confirm that our non-item modeling can improve the performance of several different recommender models in different frameworks.

7 Conclusion

The effect of the inclusion of item and user properties for sequential item recommendation has been studied, but non-item interactions have been an afterthought, although they are present in most recommendation setups. In our paper, we closed this gap by giving an overview on the various types of non-item interactions and proposing different modeling approaches for them. By analyzing the datasets with HypTrails, we showed that non-item pages are useful indicators for following item interactions. We adapted several sequential recommender baselines in the popular RecBole framework to include non-item representations either by attributes or pre-computed embeddings. Evaluation on one artificially constructed and two real-life datasets showed that non-item interactions can significantly improve sequential recommendation. While all models can profit from including non-items, the degree of improvement varies between models and is dependent on several factors. One factor is the ability of models to handle noisy data, and another the selection of meaningful non-item pages with content related to the item. Our study also highlights the importance of choosing a suitable representation for non-item interactions for each dataset and scenario individually. For future work, we plan to investigate alternative fusion functions for incorporating page embeddings within the embedding layer. Additionally, applying our approach to conversational recommendation systems might be a promising direction for future research.

Acknowledgments

This work is partially supported by both the German Research Foundation (DFG) under grant number 438232455 (HydrAS) and the German Federal Ministry of Education and Research (BMBF) under grant number 01IS22051B (KLiMod). The authors are responsible for the content of this publication.

References

- [1] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-Commerce Recommendation in Alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*. ACM. <https://doi.org/10.1145/3326937.3341261>
- [2] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. *CoRR* abs/2003.10555 (2020). <https://arxiv.org/abs/2003.10555> arXiv: 2003.10555.
- [3] Alexander Dallmann, Daniel Zoller, and Andreas Hotho. 2021. A Case Study on Sampling Strategies for Evaluating Neural Sequential Item Recommendation Models. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)*. Association for Computing Machinery, New York, NY, USA, 505–514. <https://doi.org/10.1145/3460231.3475943>
- [4] Gabriel de Souza Pereira Moreira, Sara Rabhi, Jeong Min Lee, Ronay Ak, and Even Oldridge. 2021. Transformers4Rec: Bridging the Gap between NLP and Sequential / Session-Based Recommendation. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)*. Association for Computing Machinery, New York, NY, USA, 143–153. <https://doi.org/10.1145/3460231.3474255> event-place: Amsterdam, Netherlands.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, Minneapolis, Minnesota, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- [6] Xinyan Fan, Zheng Liu, Jianxun Lian, Wayne Zhao, Xing Xie, and Ji-Rong Wen. 2021. Lighter and Better: Low-Rank Decomposed Self-Attention Networks for Next-Item Recommendation. 1733–1737. <https://doi.org/10.1145/3404835.3462978>
- [7] Elisabeth Fischer, Daniel Schlör, Albin Zehe, and Andreas Hotho. 2023. Enhancing Sequential Next-Item Prediction through Modelling Non-Item Pages. In *2023 IEEE International Conference on Data Mining Workshops (ICDMW)*. 128–136. <https://doi.org/10.1109/ICDMW60847.2023.00024>
- [8] Elisabeth Fischer, Daniel Zoller, Alexander Dallmann, and Andreas Hotho. 2020. Integrating Keywords into BERT4Rec for Sequential Recommendation. In *KI 2020: Advances in Artificial Intelligence*, Ute Schmid, Franziska Klügl, and Diedrich Wolter (Eds.). Springer International Publishing, Cham, 275–282. https://doi.org/10.1007/978-3-030-58285-2_23
- [9] Elisabeth Fischer, Daniel Zoller, and Andreas Hotho. 2021. Comparison of Transformer-Based Sequential Product Recommendation Models for the Coveo Data Challenge. *SIGIR Workshop On eCommerce* (July 2021).
- [10] Balázs Hidasi and Alexandros Karatzoglou. 2018. Recurrent neural networks with top-k gains for session-based recommendations. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 843–852.
- [11] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *ICLR (Poster)*, Yoshua Bengio and Yann LeCun (Eds.).
- [12] Balázs Hidasi, Massimo Quadrana, Alexandros Karatzoglou, and Domonkos Tikk. 2016. Parallel Recurrent Neural Network Architectures for Feature-rich Session-based Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*. ACM, New York, NY, USA, 241–248. <https://doi.org/10.1145/2959100.2959167> event-place: Boston, Massachusetts, USA.
- [13] Yupeng Hou, Binbin Hu, Zhiqiang Zhang, and Wayne Xin Zhao. 2022. CORE: Simple and Effective Session-based Recommendation within Consistent Representation Space. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 1796–1801. <https://doi.org/10.1145/3477495.3531955> event-place: Madrid, Spain.
- [14] Shotaro Ishihara, Shuhei Goda, and Hidehisa Arai. 2021. Adversarial validation to select validation data for evaluating performance in e-commerce purchase intent prediction. (2021).
- [15] Akshay Jagatap, Nikki Gupta, Sachin Farfade, and Prakash Mandayam Comar. 2023. AttriBERT - Session-based Product Attribute Recommendation with BERT. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. Association for Computing Machinery, New York, NY, USA, 3421–3425. <https://doi.org/10.1145/3539618.3594714> event-place: Taipei, Taiwan.
- [16] W. Kang and J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *2018 IEEE International Conference on Data Mining (ICDM)*. IEEE Computer Society, Los Alamitos, CA, USA, 197–206. <https://doi.org/10.1109/ICDM.2018.00035>
- [17] Tobias Koopmann, Martin Becker, Florian Lemmerich, and Andreas Hotho. 2024. CompTrails: comparing hypotheses across behavioral networks. *Data Min. Knowl. Discov.* 38, 3 (Jan. 2024), 1258–1288. <https://doi.org/10.1007/s10618-023-00996-8> Place: USA Publisher: Kluwer Academic Publishers.
- [18] Jing Li, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Tao Lian, and Jun Ma. 2017. Neural Attentive Session-based Recommendation. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (CIKM '17)*. Association for Computing Machinery, New York, NY, USA, 1419–1428. <https://doi.org/10.1145/3132847.3132926> event-place: Singapore, Singapore.
- [19] Chang Liu, Xiaoguang Li, Guohao Cai, Zhenhua Dong, Hong Zhu, and Lifeng Shang. 2021. Non-invasive Self-attention for Side Information Fusion in Sequential Recommendation. <https://doi.org/10.48550/arXiv.2103.03578> arXiv:2103.03578.
- [20] Qiang Liu, Shu Wu, Diyi Wang, Zhaokang Li, and Liang Wang. 2016. Context-Aware Sequential Recommendation. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. 1053–1058. <https://doi.org/10.1109/ICDM.2016.0135>
- [21] L. Maaten and Geoffrey E. Hinton. 2008. Visualizing Data using t-SNE. *Journal of Machine Learning Research* (2008). <https://www.semanticscholar.org/paper/Visualizing-Data-using-t-SNE-Maaten-Hinton/1c46943103bd7b7a2c7be86859959a4144d1938b>

- [22] Gabriel de Souza P. Moreira, Sara Rabhi, Ronay Ak, Md Yasin Kabir, and Even Oldridge. 2021. Transformers with multi-modal features and post-fusion context for e-commerce session-based recommendation. *_eprint*: 2107.05124.
- [23] Aleksandr Petrov and Craig Macdonald. 2022. A Systematic Review and Replicability Study of BERT4Rec for Sequential Recommendation. In *Proceedings of the 16th ACM Conference on Recommender Systems (RecSys '22)*. Association for Computing Machinery, New York, NY, USA, 436–447. <https://doi.org/10.1145/3523227.3548487> event-place: Seattle, WA, USA.
- [24] Yoshihiro Sakatani. 2021. Session-based Recommendation Using an Ensemble of LSTM-and Matrix Factorization-based Models. (2021).
- [25] Philipp Singer, Denis Helic, Andreas Hotho, and Markus Strohmaier. 2015. HypTrails: A Bayesian Approach for Comparing Hypotheses About Human Trails on the Web. In *Proceedings of the 24th International Conference on World Wide Web (WWW '15)*. International World Wide Web Conferences Steering Committee. <https://doi.org/10.1145/2736277.2741080>
- [26] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management - CIKM 19*. ACM Press. <https://doi.org/10.1145/3357384.3357895>
- [27] Jacopo Tagliabue, Ciro Greco, Jean-Francis Roy, Bingqing Yu, Patrick John Chia, Federico Bianchi, and Giovanni Cassani. 2021. SIGIR 2021 E-Commerce Workshop Data Challenge. *_eprint*: 2104.09423.
- [28] Yong Kiam Tan, Xinxing Xu, and Yong Liu. 2016. Improved Recurrent Neural Networks for Session-based Recommendations. In *Proceedings of the 1st workshop on deep learning for recommender systems*, Vol. abs/1606.08117. 17–22. <http://arxiv.org/abs/1606.08117> arXiv: 1606.08117.
- [29] Jiaxi Tang and Ke Wang. 2018. Personalized Top-N Sequential Recommendation via Convolutional Sequence Embedding. In *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining (WSDM '18)*. Association for Computing Machinery, New York, NY, USA, 565–573. <https://doi.org/10.1145/3159652.3159656> event-place: Marina Del Rey, CA, USA.
- [30] Wilson L. Taylor. 1953. "Cloze procedure": a new tool for measuring readability. *Journalism & Mass Communication Quarterly* 30 (1953), 415–433.
- [31] Trinh Xuan Tuan and Tu Minh Phuong. 2017. 3D Convolutional Networks for Session-based Recommendation with Content Features. In *Proceedings of the Eleventh ACM Conference on Recommender Systems (RecSys '17)*. ACM, New York, NY, USA, 138–146. <https://doi.org/10.1145/3109859.3109900> event-place: Como, Italy.
- [32] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.
- [33] Martin Wattenberg, Fernanda Viégas, and Ian Johnson. 2016. How to Use t-SNE Effectively. *Distill* (2016). <https://doi.org/10.23915/distill.00002>
- [34] Liwei Wu, Shuqing Li, Cho-Jui Hsieh, and James Sharpnack. 2020. SSE-PT: Sequential Recommendation Via Personalized Transformer. In *Proceedings of the 14th ACM Conference on Recommender Systems (RecSys '20)*. Association for Computing Machinery, New York, NY, USA, 328–337. <https://doi.org/10.1145/3383313.3412258> event-place: Virtual Event, Brazil.
- [35] Yueqi Xie, Peilin Zhou, and Sunghun Kim. 2022. Decoupled Side Information Fusion for Sequential Recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '22)*. Association for Computing Machinery, New York, NY, USA, 1611–1621. <https://doi.org/10.1145/3477495.3531963> event-place: Madrid, Spain.
- [36] Chengfeng Xu, Pengpeng Zhao, Yanchi Liu, Jiajie Xu, Victor S.Sheng S.Sheng, Zhiming Cui, Xiaofang Zhou, and Hui Xiong. 2019. Recurrent Convolutional Neural Network for Sequential Recommendation. In *The World Wide Web Conference on - WWW'19*. ACM Press. <https://doi.org/10.1145/3308558.3313408>
- [37] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. XLNet: generalized autoregressive pretraining for language understanding. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc., Red Hook, NY, USA. Place: Red Hook, NY, USA Publisher: Curran Associates Inc..
- [38] Fajie Yuan, Alexandros Karatzoglou, Ioannis Arapakis, Joemon M. Jose, and Xiangnan He. 2019. A Simple Convolutional Generative Network for Next Item Recommendation. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining (WSDM '19)*. Association for Computing Machinery, New York, NY, USA, 582–590. <https://doi.org/10.1145/3289600.3290975> *_eprint*: 1808.05163.
- [39] Albin Zehe, Elisabeth Fischer, Jonas Kaiser, Toni Wagner, and Andreas Hotho. 2024. Adapting Sequential Recommender Models to Content Recommendation in Chat Data using Non-Item Page-Models. *Proceedings of the Sixth Knowledge-aware and Conversational Recommender Systems Workshop* (Oct. 2024). <https://ceur-ws.org/Vol-3817/>
- [40] Qi Zhang, Longbing Cao, Chongyang Shi, and Zhendong Niu. 2022. Neural Time-Aware Sequential Recommendation by Jointly Modeling Preference Dynamics and Explicit Feature Couplings. *IEEE Transactions on Neural Networks and Learning Systems* 33, 10 (2022), 5125–5137. <https://doi.org/10.1109/TNNLS.2021.3069058>
- [41] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, Yingqian Min, Zhichao Feng, Xinyan Fan, Xu Chen, Pengfei Wang, Wendi Ji, Yaliang Li, Xiaoling Wang, and Ji-Rong Wen. 2021. RecBole: Towards a Unified, Comprehensive and Efficient Framework for Recommendation Algorithms. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management (CIKM '21)*. ACM, 4653–4664. <https://doi.org/10.1145/3459637.3482016>

Table 7. Hitrate and NDCG on the Random-SynDS dataset with Page Embeddings. The datasets with growing percentages of randomization for non-items are denoted as “X%R”

Model	Metrics	0%R	10%R	20%R	30%R	40%R	50%R	60%R	70%R	80%R	90%R	100%R
BERT4Rec	HR@1	.092	.006	.006	.011	.005	.005	.005	.007	.007	.003	.005
	HR@5	.189	.019	.018	.041	.018	.018	.019	.022	.023	.015	.018
	HR@10	.231	.035	.032	.071	.031	.031	.036	.038	.041	.030	.032
	NDCG@5	.143	.012	.012	.026	.012	.012	.012	.014	.015	.009	.011
	NDCG@10	.156	.017	.016	.035	.016	.016	.017	.019	.021	.014	.016
Caser	HR@1	.288	.211	.175	.162	.133	.043	.023	.019	.017	.016	.018
	HR@5	.546	.448	.369	.331	.275	.116	.066	.060	.055	.053	.059
	HR@10	.653	.544	.455	.407	.342	.166	.102	.095	.087	.083	.090
	NDCG@5	.424	.335	.277	.250	.207	.080	.044	.039	.036	.035	.038
	NDCG@10	.459	.367	.305	.275	.229	.096	.056	.051	.046	.045	.048
Core	HR@1	.310	.297	.252	.191	.111	.065	.034	.021	.016	.015	.014
	HR@5	.551	.514	.443	.359	.259	.182	.118	.081	.064	.062	.062
	HR@10	.646	.595	.516	.430	.328	.250	.178	.133	.110	.107	.107
	NDCG@5	.437	.413	.353	.280	.188	.125	.077	.051	.040	.038	.037
	NDCG@10	.468	.439	.377	.302	.210	.147	.096	.068	.054	.053	.052
GRU4Rec	HR@1	.368	.337	.296	.257	.221	.173	.115	.041	.024	.020	.023
	HR@5	.625	.563	.497	.428	.373	.301	.220	.108	.072	.068	.070
	HR@10	.721	.646	.575	.493	.431	.352	.269	.155	.113	.107	.110
	NDCG@5	.504	.458	.402	.349	.302	.241	.170	.075	.048	.044	.046
	NDCG@10	.536	.484	.428	.369	.321	.258	.185	.090	.061	.057	.059
LightSANs	HR@1	.351	.316	.284	.242	.208	.177	.126	.072	.032	.016	.017
	HR@5	.613	.547	.488	.419	.363	.307	.229	.148	.083	.048	.049
	HR@10	.706	.633	.568	.485	.422	.357	.275	.187	.117	.073	.077
	NDCG@5	.491	.439	.393	.336	.291	.246	.180	.111	.058	.032	.033
	NDCG@10	.521	.467	.418	.358	.310	.262	.195	.124	.069	.040	.042
NARM	HR@1	.362	.332	.297	.252	.216	.167	.107	.035	.022	.022	.025
	HR@5	.621	.559	.502	.430	.374	.299	.209	.104	.072	.075	.073
	HR@10	.711	.641	.573	.496	.434	.355	.261	.154	.114	.117	.114
	NDCG@5	.499	.454	.405	.347	.300	.237	.161	.070	.047	.049	.049
	NDCG@10	.529	.480	.428	.368	.320	.255	.178	.086	.061	.062	.062
NextItNet	HR@1	.264	.231	.131	.083	.035	.018	.011	.011	.008	.011	.011
	HR@5	.483	.428	.293	.194	.090	.057	.038	.033	.032	.035	.040
	HR@10	.575	.505	.368	.258	.132	.089	.062	.054	.053	.056	.063
	NDCG@5	.380	.335	.215	.141	.062	.038	.024	.022	.020	.023	.026
	NDCG@10	.410	.360	.240	.161	.076	.048	.032	.029	.027	.029	.033
SASRec	HR@1	.367	.347	.302	.262	.229	.182	.118	.071	.031	.021	.022
	HR@5	.628	.576	.510	.435	.382	.317	.227	.153	.090	.071	.069
	HR@10	.719	.657	.584	.503	.441	.372	.278	.201	.137	.111	.112
	NDCG@5	.506	.468	.412	.354	.311	.253	.175	.113	.061	.046	.046
	NDCG@10	.536	.495	.436	.376	.330	.271	.192	.129	.076	.059	.060

Table 8. Hitrate and NDCG on the Random-SynDS dataset with ID Embeddings. The datasets with growing percentages of randomization for non-items are denoted as “X%R”

Model	Metrics	0%R	10%R	20%R	30%R	40%R	50%R	60%R	70%R	80%R	90%R	100%R
BERT4Rec	HR@1	.126	.094	.084	.065	.063	.004	.004	.004	.004	.002	.004
	HR@5	.249	.181	.170	.138	.133	.016	.014	.012	.012	.007	.011
	HR@10	.303	.225	.214	.173	.162	.027	.023	.020	.019	.014	.018
	NDCG@5	.191	.140	.129	.103	.100	.010	.009	.008	.008	.005	.008
	NDCG@10	.209	.154	.143	.114	.109	.014	.012	.010	.010	.007	.010
Caser	HR@1	.318	.275	.237	.180	.099	.052	.022	.018	.017	.018	.018
	HR@5	.572	.493	.433	.338	.212	.133	.069	.056	.057	.061	.057
	HR@10	.674	.581	.510	.408	.273	.180	.104	.090	.089	.092	.091
	NDCG@5	.452	.391	.340	.263	.158	.094	.046	.037	.037	.039	.038
	NDCG@10	.485	.419	.365	.286	.178	.109	.058	.048	.047	.050	.049
Core	HR@1	.348	.324	.282	.224	.175	.130	.089	.051	.030	.014	.016
	HR@5	.602	.552	.491	.417	.350	.282	.209	.137	.094	.063	.059
	HR@10	.697	.633	.565	.485	.414	.342	.268	.194	.143	.107	.102
	NDCG@5	.484	.445	.393	.327	.268	.211	.152	.095	.062	.038	.037
	NDCG@10	.515	.471	.417	.349	.289	.230	.171	.113	.078	.052	.051
GRU4Rec	HR@1	.356	.327	.294	.251	.216	.177	.121	.037	.021	.018	.019
	HR@5	.613	.558	.501	.422	.368	.304	.220	.099	.069	.059	.064
	HR@10	.706	.635	.572	.484	.423	.353	.270	.142	.105	.096	.100
	NDCG@5	.494	.450	.404	.342	.297	.244	.174	.069	.045	.039	.041
	NDCG@10	.524	.475	.428	.363	.315	.260	.190	.082	.057	.050	.053
LightSANs	HR@1	.344	.313	.280	.239	.214	.182	.142	.103	.060	.018	.014
	HR@5	.607	.550	.486	.421	.373	.313	.249	.183	.117	.055	.045
	HR@10	.702	.633	.564	.490	.432	.366	.293	.221	.150	.083	.073
	NDCG@5	.483	.439	.389	.336	.299	.252	.199	.145	.090	.036	.029
	NDCG@10	.514	.466	.414	.358	.318	.269	.213	.157	.101	.045	.038
NARM	HR@1	.355	.326	.289	.251	.213	.164	.112	.037	.019	.019	.019
	HR@5	.615	.556	.492	.426	.368	.294	.212	.099	.069	.066	.063
	HR@10	.709	.640	.570	.493	.426	.346	.264	.142	.109	.100	.104
	NDCG@5	.493	.449	.397	.344	.295	.232	.164	.068	.044	.043	.041
	NDCG@10	.524	.476	.422	.366	.314	.249	.181	.082	.057	.054	.054
NextItNet	HR@1	.256	.201	.150	.090	.043	.030	.012	.010	.010	.008	.011
	HR@5	.475	.384	.309	.208	.102	.081	.040	.035	.036	.034	.035
	HR@10	.566	.465	.384	.269	.139	.115	.064	.055	.060	.055	.062
	NDCG@5	.372	.297	.233	.151	.074	.057	.026	.023	.023	.021	.023
	NDCG@10	.401	.324	.257	.171	.086	.067	.033	.029	.031	.028	.032
SASRec	HR@1	.367	.344	.303	.255	.226	.196	.149	.112	.064	.026	.021
	HR@5	.631	.578	.515	.436	.389	.330	.266	.197	.133	.080	.067
	HR@10	.725	.662	.589	.505	.448	.380	.313	.239	.171	.118	.105
	NDCG@5	.508	.469	.416	.351	.312	.267	.211	.157	.100	.053	.044
	NDCG@10	.539	.496	.440	.374	.332	.283	.226	.170	.112	.066	.056

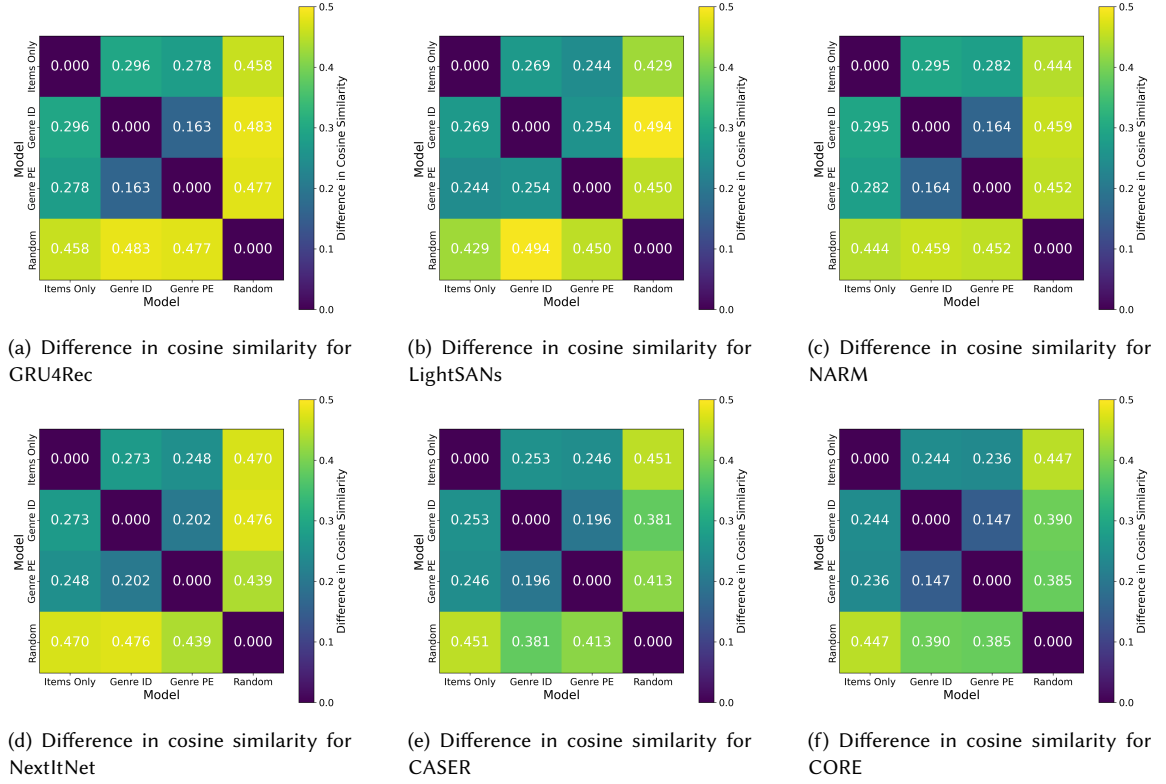


Fig. 15. Difference in Item-to-Item cosine similarity between different model variants. For each model the cosine similarity between all movies is computed and normalized. We report the average of the absolute difference in similarity between two models. A lower score means item-item relations are similar in both models. A randomly generated embedding is included as a baseline.

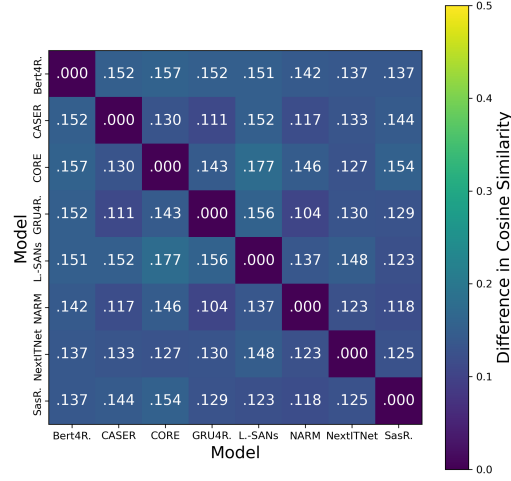


Fig. 16. Difference in Item-to-Item cosine similarity between different models with CPID on Prev-SynDS. We report the average of the absolute difference in similarity between two models. A lower score means item-item relations are similar in both models. A randomly generated embedding is included as a baseline.

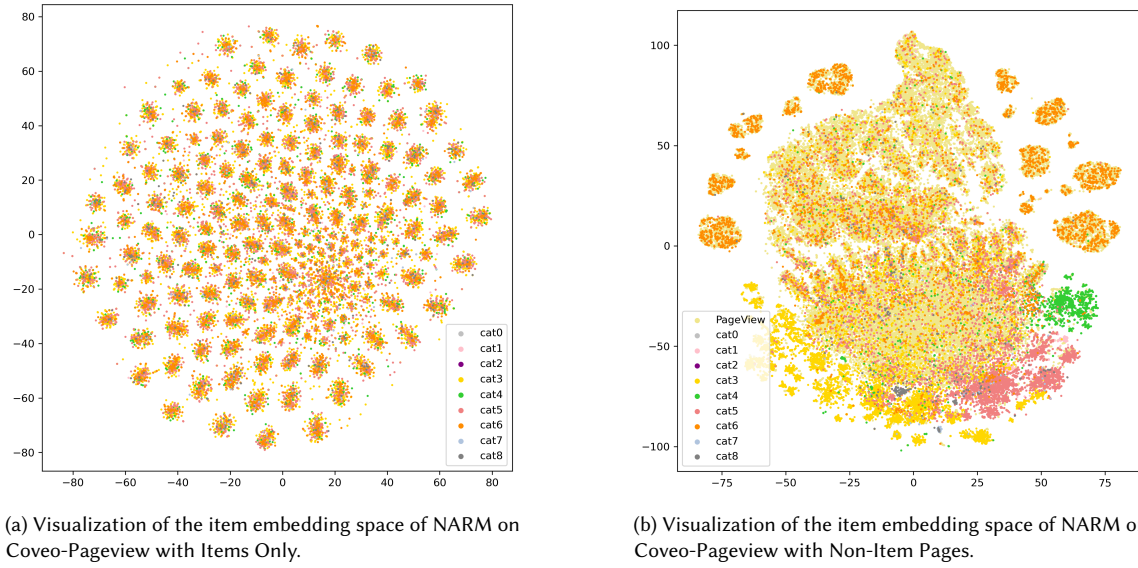
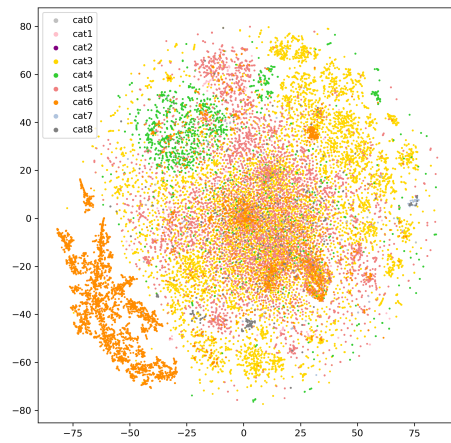
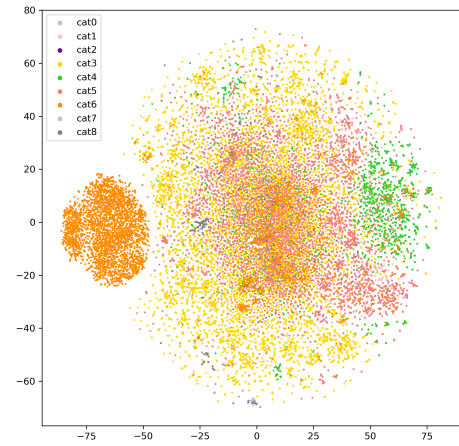


Fig. 17. t-SNE visualization of item embedding space of NARM on Coveo-Pageview with ID embeddings.



(a) Visualization of the item embedding space of SAS-Rec on Coveo-Search with Items Only.



(b) Visualization of the item embedding space of SAS-Rec on Coveo-Search trained with the embedded search query.

Fig. 18. t-SNE visualization of item embedding space of SASRec on Coveo-Search.